ALGORITMI II · A.A. 2019-2020

I
- PARADIGMI DI SVILUPPO DI ALGORITMI
  - Programmazione Dinamica
  - Algoritmi Greedy
  - Algoritmi estendenti su Grafi Pesati
    - Minimo Albero di Copertura + Applicazioni
    - Percorsi minimi in grafi pesati
    - Flusso su reti + Applicazioni

II
- PROBLEMI INTRATTABILI
  - Riduzioni Polinomiali
  - Le classi di complessità P e NP
  - Problemi NP-Completi e Intrattabilità
- Tecniche Approssimanti } Problemi Intrattabili
- Tecniche Euristiche

LIBRI DI TESTO:
- J. KLEINBERG, E. TARDOS "ALGORITHM DESIGN"
- TH. CORMEN et Al. "INTRODUCTION TO ALGORITHMS"

SOLUZIONE DI PROBLEMA

1) FORMALIZZAZIONE (FORMULAZIONE MATEMATICA)
   - PARAMETRI DI INGRESSO: quali sono, che forma hanno
   - SOLUZIONE: forma e proprietà

2) DEFINIZIONE PROCEDURA DI SOLUZIONE
   - Seguendo Principi di Decomposizione e/o Riduzione
   - Applicando uno dei paradigmi sopra (greedy, prog. dinamica, divide et impera, ...)

3) STABILIRE CORRETTEZZA

4) STABILIRE LIMITE SUPERIORE AL TEMPO DI ESECUZIONE
   - Precisare STRUTTURE DATI SPECIFICHE per ottenere tempi ottimali

---

PROBLEMA: MATCHING STABILE

"Progettare un sistema di reclutamento che è autosostenuto"
- 2 gruppi distinti: Aziende e Lavoratori; Ospedali e Tirocinanti; ecc...
- Ogni membro di un gruppo ha delle preferenze sui membri dell'altro gruppo
- Si vogliono accoppiare membri dei due gruppi in modo che nessuno possa decidere di modificare il proprio accoppiamento

PROBLEMA: $x \leftrightarrow y$, $x' \leftrightarrow y'$ MA { x preferisce y' a y; y' preferisce x a x' } $\Rightarrow$ x e y' si accoppiano perché entrambi insoddisfatti

OBIETTIVO: Processo che renda ogni cambio di coppia svantaggioso per uno dei 2 membri coinvolti (x e y)

1) FORMALIZZAZIONE DEL PROBLEMA

DATI D'INGRESSO
- 2 gruppi della stessa dimensione X e Y con $|X| = |Y| = n$
- $\leq_x \subset Y \times Y$: relazione di preferenza di $x \in X$
- $\leq_y \subset X \times X$: relazione di preferenza di $y \in Y$

$y_2 <_x y_1$: x preferisce y_1 a y_2
$x_1 <_y x_2$: y preferisce x_1 a x_2

SOLUZIONE

MATCHING $M = \{(x,y) \mid x \in X \text{ e } y \in Y\}$ t.c. se $(x_1,y_1) \in M$ e $(x_2,y_2) \in M$ allora $x_1 = x_2 \Leftrightarrow y_1 = y_2$

[M è il grafo di una funzione iniettiva (parziale) da X a Y]

MATCHING PERFETTO: $|M| = |X| = |Y| = n$

[M è il grafo di una funzione biiettiva tra |X| e |Y|]

INSTABILITÀ IN M

$(x,y) \notin M$ t.c. se $(x,y') \in M$ e $(x',y) \in M$ allora $y <_x y'$ e $x <_y x'$

"x e y preferiscono accoppiarsi tra loro che rimanere con i partner attuali"

MATCHING STABILE: MATCHING PERFETTO per cui NON ESISTONO INSTABILITÀ

M è STABILE sse $\forall (x,y) \in M$ $\forall (x',y') \in M$ $y <_x y'$ oppure $x' <_{y'} x$

"Almeno uno tra x e y preferisce il/a attuale partner a quello in (x,y)"

(x,y) non è una instabilità

ESEMPIO: $X = \{x_1, x_2\}$ e $Y = \{y_1, y_2\}$ $|X \times Y| = 4$, $|M| = 2$, $4 - 2$

x_1	y_1	y_2
x_2	y_1	y_2

y_1	x_1	x_2
y_2	x_1	x_2

$M = \{(x_1,y_2), (x_1,y_1)\}$ — STABILE

$Pr(x_1) = \{y_1\}$

INSTABILITÀ?
- (x_1,y_2) NO!
- (x_2,y_2) NO!

$Pr(x_2) = \{y_2\}$ INSTABILE

$M = \{(x_1,y_2), (x_2,y_1)\}$

INSTABILITÀ?
- (x_1,y_1) SÌ!
- ~~(x_2,y_2)~~ NO!

ESEMPIO:

x_1	y_1	y_2
x_2	y_1	y_2

y_1	x_1	x_2
y_2	x_1	x_2

$M = \emptyset$ $\{(x_1,y_1), (x_2,y_2)\}$

INSTABILITÀ?
- (x_1,y_2) NO!
- (x_2,y_2) NO!

$M = \{(x_1,y_2), (x_2,y_1)\}$ — STABILE

$Pr(x_1) = \{y_1, y_2\}$

INSTABILITÀ?
- (x_1,y_1) NO!
- (x_2,y_1) NO!

$Pr(x_2) = \{y_1, y_2\}$ $M = \{(x_1,y_2), (x_2,y_1)\}$

$Best(x_1) = y_1$ STABILE

$Best(x_2) = y_2$

2) Definizione di una soluzione algoritmica

- Quanti matching esistono tra X e Y se $|X| = |Y| = n$?

> Una PERMUTAZIONE di $N = \{1, \dots, n\}$ è una funzione biiettiva da N a N

Considerando $X = Y = \{1, \dots, n\} = N$, un matching è una permutazione di N

Vi sono quindi $n!$ matching diversi.

Cercare una soluzione generando e verificando la stabilità di ogni matching costerebbe $\Omega(n!) = \Omega(n^n) = \Omega(2^{n \log n})$ $\left(2^{\log n}\right)^n$

Verificare la stabilità richiede $\Theta(n^2)$

Algoritmo di Gale-Shapley

MATCH $(X, Y, \{<_x\}_{x \in X}, \{<_y\}_{y \in Y})$

- $M = \emptyset$
- WHILE (esiste $x \in X$ libero che non si è proposto a tutti gli $y \in Y$) DO
  - SCEGLI UN TALE x
  - SIA y IL PREFERITO CUI x NON HA PROPOSTO
  - IF y è libero THEN
    - $M = M \cup \{(x, y)\}$
  - ELSE
    - SIA x' t.c. $(x', y) \in M$
    - IF $x <_y x'$ THEN
      - $M = (M \setminus \{(x', y)\}) \cup \{(x, y)\}$
      - x' diventa libero
    - ELSE
      - x rimane libero

1) L'Algoritmo G-S termina sempre? Se sì quante iterazioni massimo?

2) L'M restituito da G-S è sempre un MATCHINGSTABILE ?

1) L'Algoritmo G-S termina sempre? Se sì, quante iterazioni massimo?

MISURA DI PROGRESSO: misura quanto vicini siamo alla terminazione.

Detto "i" l'indice di una iterazione:

$Pr(i)$: n° di coppie (x, y) t.c. x ha già proposto a y entro la i-esima iterazione

$$Pr(i+1) > Pr(i), \quad \forall i \geq 0$$

Quante coppie (x, y) esistono? n^2

Al massimo ci possono essere n^2 proposte diverse

- Potremmo usare il n° di x liberi all'iterazione (i) come misura di progresso $Pr(i)$?

$Pr(i+1) < Pr(i)$? NO!

- Potremmo usare il n° di coppie in M all'iterazione (i) come misura di progresso $Pr(i)$?

$Pr(i+1) > Pr(i)$? NO!

2) L'M restituito da G-S è sempre un MATCHING STABILE?

- L'M restituito da G-S è un **MATCHING PERFETTO**

M è un **MATCHING** (fu. iniettiva)?

OK!

M è **PERFETTO**? Per assurdo

Assumiamo $\exists\, x \in X$ t.c. $(x,y) \notin M, \forall y \in Y$

al termine di G-S.

$\Rightarrow$ x si è proposto a tutti gli y?

- No ... IMPOSSIBILE

- Sì ... $|Y| = n$

- L'M restituito da G-S è un **MATCHING STABILE**

Per assurdo. Sia $(x,y) \notin M$ t.c.

$(x,y') \in M$ e $(x',y) \in M$ e $y <_x y'$ e $x <_y x'$

(a) (b)

2 casi:

1) x si propose a y' prima che a y

2) x si propose a y prima che a y'

- Caso 1 è impossibile a causa di (a)

- Caso 2) $\Rightarrow$ y rifiuta x
  - o subito [sta con $x'' <_y x$]
  - o più avanti, quando un $x'' <_y x$ si propone a y

Alla fine $(x',y) \in M \Rightarrow x' \leq_y x'' <_y x$

che contraddice (b)

G-S è sotto-specificato:

- Non è determinata la scelta dell'x libero da selezionare ad ogni iterazione.

Può questo NON-DETERMINISMO influenzare la soluzione restituita?

- NO! Ogni possibile esecuzione di G-S restituisce lo stesso MATCHING

Quale MATCHING viene restituito?

$Pr(x)$: insieme dei PARTNER VALIDI per x

$$Pr(x) = \{ y \in Y \mid \exists M.\ M \text{ è matching stabile e } (x,y) \in M \}$$

$Best(x)$: MIGLIOR PARTNER VALIDO per x

$$Best(x) = y \quad \text{sse} \quad \forall y' \in Pr(x).\ y' \neq y \Rightarrow y <_x y'$$

$Best(x)$ è il partner valido per x che x preferisce, tra tutti i suoi partner validi

Teorema: Ogni esecuzione di G-S restituisce

$$M^* = \{ (x, Best(x)) \mid x \in X \}$$

Dimostrazione: Per assurdo.

Sia π un'esecuzione di G-S che restituisce $M \neq M^*$. Quindi

$$\exists x \text{ t.c. } (x,\bar{y}) \in M \text{ e } \bar{y} \neq Best(x)$$

Poiché tutti gli x fanno le proposte in ordine decrescente di preferenza, x deve essere stato rifiutato dal suo MIGLIOR PARTNER VALIDO $Best(x)$

Scegliamo come x il primo elemento di X che viene rifiutato da un PARTNER VALIDO e sia t l'istante in cui ciò avviene

Ovviamente x viene rifiutato da $y = Best(x)$ all'istante t (PERCHÉ??)

Perché x viene rifiutato da y al tempo t?

- Perché y è occupato con x' al tempo t e $x' <_y x$. (1)

- Ma $y = Best(x) \in Pr(x)$, quindi

$$\exists M'.\ (x,y) \in M' \text{ e } (x',y') \in M' \quad (2)$$
$$y' \in Pr(x') \text{ e } M' \text{ è MATCHING STABILE}$$

In π al tempo t, x' non è ancora mai stato rifiutato da un PARTNER VALIDO (il primo è x per ossunzione)

Quindi x' non ha ancora proposto a y' al tempo t. Ma allora $y <_{x'} y'$. (3)

Ma (1), (2) e (3) ci dicono che:

$\exists M'$. M' è MATCHING STABILE in cui $(x,y) \in M'$, $(x',y') \in M'$, $x' <_y x$ e $y <_{x'} y'$.

Ma quindi (x',y) è una **instabilità** per M' $\Rightarrow$ CONTRADDIZIONE !!!

IMPLEMENTAZIONE EFFICIENTE

1) Trovare in tempo costante un x libero

2) Dato $x \in X$, trovare in tempo costante il miglior y a cui x non si è ancora proposto

3) Dato y, trovare l'$x' \in X$ a cui è attualmente accoppiato, in tempo costante

4) Verificare in tempo costante se, dato un $y \in Y$, $x <_y x'$

---

1) Code di x liberi

2) Vettore NEXT[] (NEXT[x] è il prossimo y)

3) Vettore ACCOPPIATO[] (ACCOPPIATO[y] è l'x attualmente accoppiato a y)

4) pos[y][x] posizione di x nella lista di preferenze di y

$$pos[y][x] < pos[y][x'] \;?$$

Queste strutture possono essere popolate in fase preliminare.

Assumiamo che $pref_x : \{1 \dots |Y|\} \to Y$ rappresenti la lista di preferenze di x ($pref_y : \{1 \dots |X|\} \to X$)

```
INIT(X, Y, {pref_x}_{x∈X}, {pref_y}_{y∈Y})
    FX = X        /* tutti gli x sono liberi */
    FOR EACH y∈Y DO
        ACCOPPIATO[y] = nil   /* nessuno è accoppiato */
        FOR EACH i∈{1..|X|} DO
            pos[y][pref_y[i]] = i
    FOR EACH x∈X DO
        NEXT[x] = 1
```

l'i-esimo x nella lista di preferenze di y

TEMPO RICHIESTO: $\Theta(n^2)$ con $n = |X| = |Y|$

```
G-S(X, Y, {pref_x}_{x∈X}, {pref_y}_{y∈Y})
    INIT(X, Y, {pref_x}_{x∈X}, {pref_y}_{y∈Y})
    WHILE FX ≠ ∅ DO
        x = TESTA(FX)
        y = pref_x[NEXT[x]]
        NEXT[x] += 1
        IF ACCOPPIATO[y] = nil THEN
            ACCOPPIATO[y] = x
        ELSE
            x' = ACCOPPIATO[y]
            IF pos[y][x] < pos[y][x'] THEN
                ACCOPPIATO[y] = x
                FX = FX \ {x} ∪ {x'}
```

TEMPO RICHIESTO: $\Theta(n^2)$ con $n = |X| = |Y|$

PROGRAMMAZIONE DINAMICA

- Tecnica di progettazione di algoritmi
- Estende DIVIDE ET IMPERA
  - Sottoproblemi non disgiunti
  - Problemi di ottimizzazione

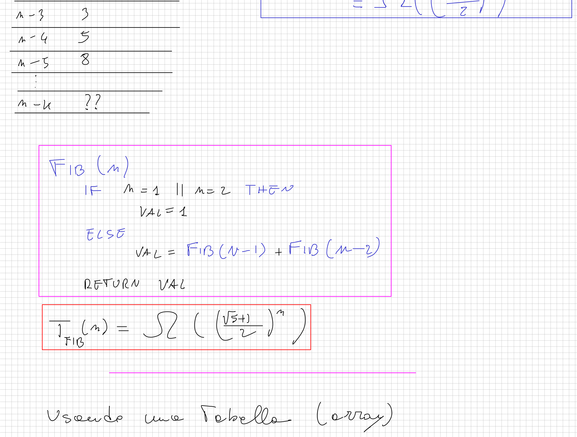
Sottoproblemi non disgiunti

- MERGESORT e QUICKSORT generano sottoproblemi disgiunti

- Funzione di FIBONACCI

$$F(n) = \begin{cases} 1 & \text{se } n = 1 \text{ o } n = 2 \\ F(n-1) + F(n-2) & \text{se } n > 2 \end{cases}$$

Albero delle chiamate ricorsive:



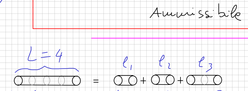
n	1
n-1	1
n-2	2
n-3	3
n-4	5
n-5	8
⋮	
n-k	??

$$Fib(n) = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}} = \Omega\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right)$$

```
FIB (n)
  IF  n = 1 || n = 2  THEN
      VAL = 1
  ELSE
      VAL = FIB(n-1) + FIB(n-2)
  RETURN VAL
```

$$T_{FIB}(n) = \Omega\left(\left(\frac{\sqrt{5}+1}{2}\right)^n\right)$$

Usando una Tabella (array)




```
FIB(n)
  IF  f_n < 0  THEN
      IF  n = 1 || n = 2  THEN
          VAL = 1
      ELSE
          VAL = FIB(n-1) + FIB(n-2)
      f_n = VAL
  RETURN f_n
```

```
FIB(n)
  f_1 = f_2 = 1
  FOR i = 3 TO n DO
      f_i = f_{i-1} + f_{i-2}
  RETURN f_n
```

$$T_{FIB}(n) = \Theta(n)$$

Problema: TAGLIO OTTIMO DEL TRONCO

- Un tronco d'albero di lunghezza L
- $\forall\, \ell \le L$, p_ℓ è il prezzo di un taglio del tronco di lunghezza $\ell \le L$

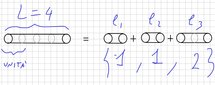
Soluzione: Qual è l'insieme di Tagli del Tronco che rende massimo il guadagno?

INSIEME di TAGLI: $S = \{\ell_1, \ell_2, \dots, \ell_k\}$ t.c.

$\ell_1 + \ell_2 + \dots + \ell_k = L$ (Soluzione Ammissibile) e $C_S = \sum_{i=1}^{k} p_{\ell_i}$ è MASSIMO (Valore (costo) della Soluzione)

$\ell_i > 0$ $\forall\, i \in \{1, \dots, k\}$

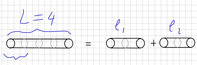
ESEMPIO: $L = 4$



$\{1, 1, 2\}$

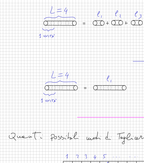
$T_1 = 1 + 1 + 2 = 4$

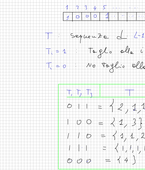
Costo$(T_1) = p_1 + p_1 + p_2$



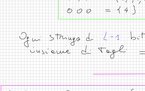
$T_2 = 2 + 2 = 4$

Costo$(T_2) = p_2 + p_2$





Quanti possibili modi di tagliare un tronco di lunghezza L esistono?



T = sequenza di $L-1$ bit; T_i sia il bit i-esimo
T_{i+1} = Taglio alla i-esima unità di misura
$T_i = 0$ = No taglio alla i-esima " " "

T_1 T_2 T_3	T
0 1 1	$\{1, 1+2\}$
1 0 0	$\{1, 3\}$
1 1 0	$\{1, 2, 2\}$
1 1 1	$\{1, 1, 1, 1\}$
0 0 0	$\{4\}$

Ogni STRINGA di $L-1$ bit corrisponde a un possibile insieme di Tagli $\Rightarrow$ 2^{L-1} possibili soluzioni

1) STUDIO STRUTTURA SOLUZIONI

Insieme di Tagli $T = \{t_1, t_2, \dots, t_k\}$ t.c.

$t_1 + t_2 + \dots + t_k = L$ e $P_{t_1} + P_{t_2} + \dots + P_{t_k}$ è MASSIMO

Soluzione Ammissibile — Valore (costo) della Soluzione

Decomponiamo una Soluzione ammissibile per $P(L)$ in Sottosoluzioni per suoi Sottoproblemi

$S = \{l_1, l_2, \dots, l_m\} = \{l_1\} \cup S'$ con $S' = \{l_2, l_3, \dots, l_m\}$

dove S è soluzione del problema $P(L)$

S' è soluzione del problema $P(L-l_1)$

Teorema: Se $S = \{l_1, l_2, \dots, l_m\}$ è soluzione ammissibile del prob. $P(L)$, allora $S' = \{l_2, \dots, l_m\}$ è soluzione ammissibile del problema $P(L-l_1)$

Dim.
S è ammissibile se $l_1 + l_2 + \dots + l_m = L$
$\Rightarrow$ $l_2 + \dots + l_m = L - l_1$ $\Rightarrow$ S' è ammissibile per $P(L-l_1)$

$\Rightarrow$ Ogni sol ammissibile per $P(L)$ contiene la soluzione ammissibile di un sottoproblema

(*)

Indichiamo con $S(L)$ una sol ammissibile per il problema $P(L)$

Teorema
Se $S = \{l_1, l_2, \dots, l_m\}$ è sol ammissibile per $P(L)$ e $S' = \{l_2, \dots, l_m\}$ è ottimale per sol ammissibile per $P(L-l_1)$, allora $S^* = \{l_1\} \cup S' = \{l_1, l_2', \dots, l_p'\}$ è sol. ammissibile per $P(L)$.

$\Rightarrow$ La composizione della prima scelta di una sol ammissibile per $P(L)$ e una qualsiasi sol ammissibile del sottoproblema è ancora una sol ammissibile per $P(L)$.

(*)

SOTTOSTRUTTURA OTTIMA (*)

Teorema: Se $S(L) = \{l_1, l_2, \dots, l_m\}$ è soluzione ottima per $P(L)$, allora $S'(L-l_1) = \{l_2, \dots, l_m\}$ è soluzione ottima per $P(L-l_1)$

Dim. Per assurdo. Assumiamo $S(L)$ ottima per $P(L)$ ma $S'(L-l_1)$ NON ottima per $P(L-l_1)$

$\Rightarrow$ Una soluzione ottima del problema $P(L)$ contiene soluzioni ottime di sottoproblemi di $P(L)$.

(*)

2) DEFINIZIONE RICORSIVA DEL VALORE (COSTO) DI UNA SOLUZIONE OTTIMA

- Una soluzione ammissibile per $P(L)$ ha la forma $S(L) = \{l_1, l_2, \dots, l_m\}$
- $S(L)$ è ottima se $CLS(L) = P_{l_1} + P_{l_2} + \dots + P_{l_m}$ è massimo
- $S'(L-l_1) = \{l_2, \dots, l_m\}$ è ottima per $P(L-l_1)$
- Quali sono i possibili valori per l_2?
  $l_2 \in \{1, 2, \dots, L\}$
- Se $C(L)$ denota il costo di una soluzione ottima per $P(L)$, allora
- Possiamo ottenere $C(L)$ come

$$C(L) = \max_{l_1 \in \{1, \dots, L\}} \{ P_{l_1} + C(L - l_1) \}$$

[ponendo $C(0) = 0$]

In generale, il costo della soluzione ottima al problema $P(\ell)$ è espresso dalla ricorrenza

$$C(\ell) = \begin{cases} 0 & \text{se } \ell = 0 \\ \max_{i \in \{1, \dots, \ell\}} \{ P_i + C(\ell - i) \} & \text{altrimenti} \end{cases}$$

che può essere calcolata dall'algoritmo

```
TAGLIO_OTTIMO (L)
  IF L=0 THEN
     V=0
  ELSE
     V=-∞
     FOR i=1 TO L DO
        V=MAX(V, P_i + TAGLIO_OTTIMO(L-i))
  RETURN V
```

Quante chiamate ricorsive esegue questo algoritmo ricorsivo?

$$T(L) = \begin{cases} 1 & \text{se } L=0 \\ 1 + \sum_{\ell=1}^{L} T(L-\ell) & \text{altrimenti} \end{cases}$$

$$T(L) = 1 + \sum_{\ell=0}^{L-1} T(\ell) \qquad T(L-1) = 1 + \sum_{\ell=0}^{L-2} T(\ell)$$

$$T(L) = T(L-1) + \underbrace{1 + \sum_{\ell=0}^{L-1} T(\ell)}_{2 \cdot T(L-1)} = 2 \cdot T(L-1)$$

L'albero indotto da queste espansioni è un albero binario pieno di altezza L

Il valore $T(L)$ è il n° di foglie di quest'albero

$$T(L) = 2^L$$

Tempo è esponenziale sulle Foglie L del problema

Molte chiamate ricorsive risolvono gli stessi problemi in rami diversi dell'albero di ricorrenza dell'algoritmo

Infatti è immediato osservare che $P(L)$ ha al massimo L diversi sottoproblemi

$P(0), P(1), \dots, P(L-2), P(L-1)$

Possiamo quindi usare Tabelle (arrays) per memorizzare i valori ottimi per gli $L+1$ (sotto)problemi di $P(L)$

3) CALCOLO DEL COSTO DELLA SOLUZIONE OTTIMA

$$C(\ell) = \begin{cases} 0 & \text{se } \ell = 0 \\ \max_{i \in \{1,\dots,\ell\}} \{ p_i + C(\ell - i) \} & \text{altrimenti} \end{cases}$$

Per poter calcolare $C[\ell]$ è necessario aver già calcolato $C[i]$ per ogni $i < \ell$!!

Due Tabelle $C[\,]$ e $S[\,]$

$C[\ell]$: costo della soluzione ottima per $P(\ell)$

$S[\ell]$: prima scelta della soluzione ottima a $P(\ell)$
[se sol. ottima a $P(\ell)$ è $\{\ell_1, \ell_2, \dots, \ell_k\}$, $S[\ell] = \ell_1$]

```
TAGLIO-OTTIMO (L, P[])
  C[0] = 0
  FOR ℓ = 1 TO L DO
      v = -∞
      FOR i = 1 TO ℓ DO
          IF v < P[i] + C[ℓ - i] THEN
              v = P[i] + C[ℓ - i]
              t = i
      C[ℓ] = v
      S[ℓ] = t
```

$O(L)$

$$T_{TO}(L) = \Theta(L^2)$$

4) RICOSTRUZIONE SOLUZIONE OTTIMA

Dato l'array $S[\,]$ (scelte ottime dei sottoprob.) soluzione ottima facilmente ricostruibile.

```
PRINT_TAGLIO-OTTIMO (L, P[])
    TAGLIO_OTTIMO (L, P[])
    WHILE (L > 0) DO
        PRINT S[L]
        L = L - S[L]
```

$\Theta(L)$

$T_{PTO}(L) = \Theta(L^2)$

Stampe le scelte dei Tagli che determinato il valore massimo

⇓

Soluzione Ottima

Rappresentazioni diverse delle soluzioni possono portare a
- formulazioni differenti del problema
- soluzioni algoritmiche diverse

Es.

1) Sol $\{\ell_1, \ell_2, \dots, \ell_k\}$ con ℓ_i la lunghezza dell'i-esimo taglio in ordine temporale

$\ell_i \in \{1 \dots L\} \quad \forall\, 1 \le i \le k$

$L = \sum_{i=1}^{k} \ell_i$ (ammissibilità)

$\{\ell_1, \dots, \ell_k\} = Sol(L) \Rightarrow \{\ell_2 \dots \ell_k\} = Sol(L - \ell_1)$

$$C(\ell) = \begin{cases} 0 & \text{Se } \ell = 0 \\ \max_{1 \le i \le \ell} \{ P_i + C(\ell - i) \} & \text{se } \ell > 0 \end{cases}$$

Otteniamo un n° lineare L di sottoproblemi distinti, ogni scelta richiede tempo lineare su L!

2) Sol $= \langle \ell_1, \dots, \ell_m \rangle$ con ℓ_i dimensione del taglio i in ordine non decrescente di lunghezza.

$1 \le \ell_1$, $\ell_m \le L$

$\ell_i \le \ell_{i+1} \quad \forall\, 1 \le i \le k-1$

$L = \sum_{i=1}^{k} \ell_i$ (ammissibilità)

Es: $L = 10$ Sol $= \langle 1, 1, 2, 2, 4 \rangle$

$P(i, j)$ | Problema di Tagliare un Tronco di lunghezza j in pezzi di dimensione $\ge i$ (i = unico Taglio disponibile)

Se $\langle \ell_1, \dots, \ell_m \rangle$ è $Sol(1, L)$ allora 2 casi possibili:

1) $\langle \ell_2, \dots, \ell_m \rangle$ è $Sol(2, L)$ [nessun Taglio di dimensione 1]

2) $\langle \ell_2, \dots, \ell_m \rangle$ è $Sol(1, L-1)$ [1° Taglio ha dimensione 1]

Es:

Data $Sol(1, 10) = \langle 1, 1, 2, 2, 4 \rangle \Rightarrow$

$Sol(1, 3) = \langle 1, 2, 4 \rangle$ [caso 2]

$Sol(1, 8) = \langle 2, 2, 4 \rangle = Sol(3, 8)$

(caso 2) (caso 1)

In generale:

se $\langle \ell_1, \dots, \ell_m \rangle$ è $Sol(i, \ell)$
- $\langle \ell_2, \dots, \ell_m \rangle$ è $Sol(i+1, \ell)$ [$\ell_1 \ne i$]
- $\langle \ell_1 + i, \dots, \ell_m \rangle$ è $Sol(i, \ell - i)$ [$\ell_1 = i$]

Dato $0 \le \ell \le L$ e $1 \le i \le L+1$ possiamo definire ricorsivamente il costo di $Sol(i, \ell)$:

$$C(i, \ell) = \begin{cases} 0 & \text{se } \ell < i \\ \max\{ C(i+1, \ell), P_i + C(i, \ell - i) \} & \text{altrimenti} \end{cases}$$

$C(2,6) = \max \begin{cases} C(3,6) \\ P_2 + C(3,3) \end{cases}$

$C(4,7) = \max \begin{cases} C(5,7) \\ P_4 + C(4,3) \end{cases}$

Otteniamo un n° quadratico L^2 di sottoproblemi, ma ogni scelta richiede Tempo costante

Dobbiamo riempire la matrice $C[i,j]$

Un modo efficiente è il seguente:

Matrice riempita diagonale per diagonale

```
FOR d = 0 TO L
  FOR i = L+1 TO L+1
    ℓ = i - (L+1)
    C[i,ℓ] = 0
```
Inizializza a 0 le diagonali sotto quella principale

```
FOR d = 1 TO L DO
  FOR i = 1 TO L-(d-1) DO
    ℓ = i + d - 1
    P1 = C[i+1, ℓ]
    P2 = P[i] + C[i, ℓ-i]
    IF P1 > P2 THEN
      C[i,ℓ] = P1
      S[i,ℓ] = 0
    ELSE
      C[i,ℓ] = P2
      S[i,ℓ] = i
```
- d itera su diagonali
- i itera su righe
- ℓ indica la colonna

C viene riempita per diagonali

Il più piccolo Taglio è $> L$

Il più piccolo Taglio è $= i$

```
PRINT-SOL (L, S[,])
  ℓ = L
  i = 1
  WHILE ℓ > 0 DO
    IF S[i,ℓ] = 0 THEN
      i = i+1
    ELSE
      PRINT i
      ℓ = ℓ - i
```

Entrambe le decomposizioni portano a un algoritmo con Tempo $O(L^2)$

In alternativa:

```
FOR i = L+1 TO 1 DO
  FOR ℓ = i-1 TO L DO
    C[i,ℓ] = 0
    S[i,ℓ] = 0
```
Inizializzazione

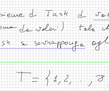
```
FOR i = L DOWNTO 1 DO
  FOR ℓ = i TO L DO
    P1 = C[i+1, ℓ]
    P2 = C[i, ℓ-i]
    IF P1 > P2 THEN
      C[i,ℓ] = P1
      S[i,ℓ] = 0
    ELSE
      C[i,ℓ] = P2
      S[i,ℓ] = i
```

SCHEDULAZIONE DI INTERVALLI PESATI

Input: Un insieme di attività (task) $T = \{1, 2, \dots, n\}$

Ogni Task i ha tempo inizio $S(i)$ e Tempo fine $F(i)$

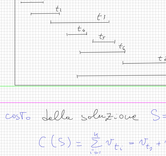
$(S(i), F(i))$ intervallo di i



Una funzione $w: T \to \mathbb{N}$ ($w(i)$ valore del Task i)

Output: Insieme di Task di valore massimo (somma dei valori) tale che nessun Task si sovrapponga agli altri

Es. $T = \{1, 2, \dots, 7\}$



Il costo della soluzione $S = \{t_{i_1}, \dots, t_{i_k}\}$ è

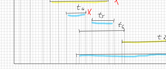
$$C(S) = \sum_{j=1}^{k} w_{t_{i_j}} = w_{t_{i_1}} + w_{t_{i_2}} + \dots + w_{t_{i_k}}$$

Soluzione $S = \{t_{i_1}, \dots, t_{i_k}\}$ per il probl. input I: $S' = \{t_{j_1}, \dots, t_{j_m}\}$ è sol. per il probl. input I'?

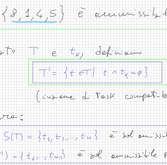
L'insieme dei Task a disposizione (intervalli) è un parametro del problema

Probl. $P(T)$ Con soluz. $S(T) = \{t_{i_1}, \dots, t_{i_k}\}$ è *ammissibile* se per ogni coppia t_i, t_j in $S(T)$ con $i \neq j$ $\quad t_i \cap t_j = \emptyset$ [no sovrapposizione]

$t \cap t' = \emptyset$ sse $F(t) < S(t') \;\vee\; F(t') < S(t)$



Es.



$\{3, 2, 1\}$ è ammissibile?

$\{2, 1, 4, 5\}$ è ammissibile?

Dato T e t_k definiamo

$$T^{t_k} = \{t \in T \mid t \cap t_k = \emptyset\}$$

(insieme di Task compatibili con t_k)

Allora:

- $S(T) = \{t_{i_1}, t_{i_2}, \dots, t_{i_k}\}$ è sol ammissibile per $P(T)$, $S(T^{t_k}) = \{t_{i_2}, \dots, t_{i_k}\}$ è sol ammissibile per $P(T^{t_k})$.

- Se t_k è parte di sol di $P(T)$ e $S'(T^{t_k}) = \{t_{i_2}, \dots, t_{i_k}\}$ è soluzione ottimale ammissibile di $P(T^{t_k})$, $\{t_{i_1}, \dots, t_{i_k}\}$ è sol ammissibile di $P(T)$???

SOTTOSTRUTTURA OTTIMA

Se $S(T) = \{t_{i_1}, t_{i_2}, \dots, t_{i_k}\}$ è Sol. ottima per $P(T)$, $S(T) = \{t_{i_2}, \dots, t_{i_k}\}$ è Sol ottima per $P(T^{t_1})$???

PROBLEMA: Quanti sono i possibili sottoproblemi di $P(T)$?

[I possibili sottoinsiemi di T]

⇓

N° esponenziale di sottoproblemi

2 CASI POSSIBILI:

1) Il problema non è risolvibile in Tempo Polinomiale

2) Le formulazione non è adeguata (può essere migliorata)

Per il problema di Schedulazione di Intervalli siamo nel caso 2

Assumiamo che i Task (intervalli) siano ordinati rispetto al Tempo di Terminazione

$T = \langle t_1, t_2, \dots, t_n \rangle$ t.c. $F(t_i) \leq F(t_j) \quad \forall\, i < j$

[possiamo ordinare T in tempo $\Theta(n \log n)$]

Definiamo una funzione

$$p(i) = \max\{\, j \in \{j, \dots, i\} \mid F(t_j) < S(t_i) \,\}$$

Se t_i è il primo task nell'ordine, $p(i)$ è l'indice di quell'ultimo Task nell'ordine che termina prima che t_i inizi.

Qual è la funzione $p(\cdot)$ dell'esempio?

Possiamo ora riformulare la decomposizione:

$P(n)$: Problema di schedulare gli n Task $\langle t_1, \dots, t_n \rangle$ ordinati per tempo di terminazione

$S(n)$: Soluzione del problema $P(n)$

1) $S(n) = \langle i_1, i_2, \dots, i_k \rangle$ è ammissibile per $P(n)$ se
   - $1 \leq i_1 < i_2 < \dots < i_k \leq n$
   - $F(t_{i_j}) < S(t_{i_{j+1}}) \quad \forall\, 1 \leq j < k$

Teo 1 Se $S(n) = \langle i_1, i_2, \dots, i_k \rangle$ è ammissibile per $P(n)$, allora vale uno dei seguenti casi:

1) $i_k = n$ e $\langle i_1, \dots, i_{k-1} \rangle$ è ammissibile per $P(p(n))$
2) $i_k < n$ e $\langle i_1, i_2, \dots, i_k \rangle$ è ammissibile per $P(n-1)$

$n \in S(n)$ → $n \notin S(n)$

Dim. Assumiamo 1) $i_k = n$ (cioè $n \in Sol(n)$) allora, per cui t_i ammissibile per $P(n)$
- $1 \leq i_1 < i_2 < \dots < i_{k-1} < i_k = n$
- $F(t_{i_j}) < S(t_{i_{j+1}}) \quad \forall\, 1 \leq j < k$

In particolare $F(t_{i_{k-1}}) < S(t_{i_k}) = S(n)$

⇓ (per definizione di $p(n)$)

$i_{k-1} \leq p(n) = p(i_k)$

ma allora $1 \leq i_1 < i_2 < \dots < i_{k-1} \leq p(i_k) = p(i_k)$

Quindi: $\langle i_1, i_2, \dots, i_{k-1} \rangle$ è ammissibile per $P(p(n)) = P(p(i_k))$

Assumo 2), allora $i_k < n$ e
- $1 \leq i_1 < i_2 < \dots < i_{k-1} < i_k < n$
- $F(t_{i_j}) < S(t_{i_{j+1}}) \quad \forall\, 1 \leq j < k$

e
- $1 \leq i_1 < i_2 < \dots < i_{k-1} < i_k < n$
- $F(t_{i_j}) < S(t_{i_{j+1}}) \quad \forall\, 1 \leq j < k$

⇓

$\langle i_1, i_2, \dots, i_k \rangle$ è ammissibile per $P(n-1)$

Teo 2 Entrambe le seguenti sono vere:

1) se $1 \le i \le m$ e $\langle i_1, \dots, i_k \rangle$ è ammissibile per $P(\ell(i))$, allora $\langle i_1, \dots, i_k, i \rangle$ è ammissibile per $P(i)$;

2) se $\langle i_1, \dots, i_k \rangle$ è ammissibile per $P(i-1)$ allora $\langle i_1, \dots, i_k \rangle$ è ammissibile per $P(i)$.

Dim. In 1), se $\langle i_1, \dots, i_k \rangle$ è ammissibile per $P(\ell(i))$, allora

$$1 \le i_1 < \dots < i_k \le \ell(i)$$
$$f[i_k] \le S[i_{j+1}] \qquad \forall\, 1 \le j < k$$

Ma $f(\ell(i)) < S(i)$ per def di $\ell(i)$

Quindi $1 \le i_1 < \dots < i_k < i_{k+1} = i$

$f[i_k] < S[i_{k+1}] \qquad \forall\, 1 \le j < k+1$

Ma allora $\langle i_1, \dots, i_k, i_{k+1} \rangle$ (in i_{k+1}) è ammissibile per $P(i)$.

Il caso 2) è ovvio (perché?)

SOTTOSTRUTTURA OTTIMA

Teo 3 Se $\langle i_1, \dots, i_{k-1}, i_k \rangle$ è sol. ottima per $P(m)$, allora una delle seguenti vale:

1) $1 \le i < m$ e $\langle i_1, \dots, i_{k-1}, i_k \rangle$ è sol. ottima per $P(m-1)$;

2) $i = m$ e $\langle i_1, \dots, i_{k-1} \rangle$ è sol. ottima per $P(\ell(m))$.

Dim. Nel caso 1), assumiamo per assurdo che $S = \langle i_1, \dots, i_k \rangle$ non sia ottima per $P(m-1)$. Ma allora $\exists\, S' = \langle i'_1, \dots, i'_{k'} \rangle$ migliore di $\langle i_1, \dots, i_k \rangle$ per $P(m-1)$, perciò S' è anche sol. ammissibile per $P(m)$, ottenendo una contraddizione con la premessa del Teorema.

Per il caso 2), assumiamo per assurdo che $S(\ell(m)) = \langle i_1, \dots, i_{k-1} \rangle$ non sia ottima per $P(\ell(m))$

$\Rightarrow \exists\, S' (\ell(m)) = \langle i'_1, \dots, i'_{k'} \rangle$ migliore

Ma per Teo precedente, $S' = \langle i'_1, \dots, i'_{k'}, m \rangle$ è ammissibile per $P(m)$

$$C(S) = v(i_1) + \dots + v(i_{k-1}) + v(m) = C(S(\ell(m))) + v(m)$$
$$< C(S'(\ell(m))) + v(m) = C(S')$$

che contraddice premessa del Teorema

2) Costo di soluzione ottima

$S(m) = \langle i_1, \dots, i_{k-1}, i_k \rangle$ ottima per $P(m)$

1) $i_k = m$ e $\langle i_1, \dots, i_{k-1} \rangle$ sol ottima per $P(\ell(m))$

$$C(S(m)) = \sum_{j=1}^{k} v(i_j) = v(i_k) + \sum_{j=1}^{k-1} v(i_j) =$$
$$= v(i_k) + C(S(\ell(i_k)))$$

Oppure

2) $i_k < m$ e $\langle i_1, \dots, i_{k-1}, i_k \rangle$ è sol ottima per $P(m-1)$.

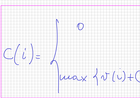
Quindi:

$$C(n) = \max\{v(n) + C(\ell(n)), C(n-1)\}$$

Funzione di costo ottimo:

$$C(i) = \begin{cases} 0 & \text{se } i = 0 \\ \max\{v(i) + C(\ell(i)), C(i-1)\} & \text{altrimenti} \end{cases}$$

3) Calcolo costo soluzione ottima



$C[i]$ dipende solo da valori $C[x]$ con $x < i$

```
Costo_Ottimo (n, v[], T[], ℓ[])
  C[0] = 0
  For i = 1 To n Do
    C[i] = C[i-1]
    If v[T[i]] + C[ℓ(i)] > C[i] Then
      C[i] = v[T[i]] + C[ℓ(i)]
```

$\Theta(n)$

4) Costruzione soluzione ottima

Il seguente algoritmo ricorsivo fornisce la sequenza ottima di attività per $P(i)$.

```
Print_Sol (i, T[], C[], ℓ[])
  If i > 0 Then
    If C[i] = C[i-1] Then
      Print_Sol (i-1, T, C)
    Else
      Print_Sol (ℓ(i), T, C)
      Print T[i]
```

Tempo di soluzione è $\Theta(n)$, assumendo che:

1) T ordinato e

2) $\ell(i)$ calcolabile in tempo $\Theta(1)$

Per garantire ciò, possiamo:

1) Ordinare T costo $\Theta(n \log n)$

2) Precalcolare $\ell()$ [in un array] a costo $O(n \log n)$ [perché??]

Quindi il costo totale di soluz. è $\Theta(n \log n)$

Per precalcolare la funzione $\ell()$ per ogni $1 \le i \le n$ possiamo usare la ricerca binaria in T.

```
Bin_Search (i, T)
  p = 1
  r = i-1
  While p ≤ r Do
    q = ⌊(p+r)/2⌋
    If f[T[q]] < s[T[i]] Then
      p = q+1
    Else
      r = q-1
  Return r
```

Per ogni task calcolare $\ell(i)$ costa $O(\log n)$

Ripempio array $\ell[.]$ con Bin_Search(i, T) per ogni task i:

```
Compute_ℓ (T, n)
  For i = 1 To n Do
    ℓ[i] = Bin_Search (i, T)
```

per un costo di $O(n \log n)$.

Massima Sottosequenza Comune

Date due sequenze

$X = x_1 \dots x_n$

$Y = y_1 \dots y_m$

trovare la **più lunga sottosequenza comune** (LCS) di X e Y

Es:

$X = A\,G\,A\,G\,C\,B\,A\,B$

$Y = C\,A\,D\,B\,D\,A\,E\,C\,A\,D$ $\quad \Rightarrow Z_1 = A\,D\,D\,A\,A\,B$ (6)

$X = A\,G\,A\,G\,C\,B\,A\,B$

$Y = C\,A\,D\,B\,D\,A\,E\,C\,A\,D$ $\quad \Rightarrow Z_2 = A\,D\,B\,E\,A\,B$ (6)

$X = A\,G\,A\,C\,D\,B\,A\,B$

$Y = C\,A\,D\,B\,D\,A\,E\,C\,A\,D$ $\quad \Rightarrow Z_3 = C\,D\,E\,A\,B$ (5)

$P(i,j)$: LCS delle sequenze $x_1 \dots x_i$ e $y_1 \dots y_j$

$X = x_1 \dots x_{n-1} x_n$

$Y = y_1 \dots y_{m-1} y_m$

$Z = z_1 \dots z_{k-1} z_k$

Sol ammissibile per $P(n,m)$

Condizione di Ammissibilità

Mettiamoci in cerca di una soluzione $Z = z_1 \dots z_{k-1} z_k$ per $P(n,m)$ costruita a un matching (parziale)

$M_Z = (i_Z, j_Z) : (i_Z, j_Z) \quad t.c.$

a) $1 \le i_Z[c] < i_Z[c+1] \le n$

b) $1 \le j_Z[c] < j_Z[c+1] \le m$

c) $\forall\, 1 \le c \le k \quad x_{i_Z[c]} = y_{j_Z[c]}$

Es: $P(9,10)$:

$X = \underset{1}{A}\,\underset{2}{G}\,\underset{3}{A}\,\underset{4}{G}\,\underset{5}{C}\,\underset{6}{B}\,\underset{7}{A}\,\underset{8}{B}\,\underset{9}{A}$

$Y = \underset{1}{C}\,\underset{2}{A}\,\underset{3}{D}\,\underset{4}{B}\,\underset{5}{D}\,\underset{6}{A}\,\underset{7}{E}\,\underset{8}{C}\,\underset{9}{A}\,\underset{10}{D}$ $\quad \Rightarrow Z = \underset{1}{A}\,\underset{2}{D}\,\underset{3}{A}\,\underset{4}{A}\,\underset{5}{B}$

$(i_Z, j_Z) = (1,2)\,(3,6)\,(5,8)\,(6,9)\,(7,10)$

$(i_Z[1], j_Z[1])\,(i_Z[2], j_Z[2])\,(i_Z[3], j_Z[3])\,(i_Z[4], j_Z[4])\,(i_Z[5], j_Z[5])$

E.C.:

$x_1 = x_2 = y_2 = A$

$x_3 = x_4 = y_6 = B$

$x_5 = y_8 = C$

$x_6 = y_9 = D$

$x_7 = y_{10} = E$

$\vdots$

1) Caratterizzazione Soluzione Ottima

$P(7,5)$

$X = A\,B\,C\,A\,B\,C$

$Y = B\,D\,A\,C\,B$ $\quad \Rightarrow Z_1 = B\,D\,B \quad Z_1 \neq X_6$

$P(i=6, j=5) = P(6,5)$

$P(7,5)$

$X = A\,B\,C\,B\,A\,B\,C$

$Y = B\,D\,A\,C\,B$ $\quad \Rightarrow Z_2 = B\,D\,C \quad Z_2 \neq Y_5$

$P(i,j) = P(7,4)$

$P(6,5)$

$X = A\,B\,C\,B\,A$

$Y = B\,D\,C\,A\,B$ $\quad \Rightarrow Z_3 = B\,C\,A\,B \quad Z_3 = X_6 = Y_5$

$P(i=6, j=4) = P(5,4)$

TEOREMA 1.

Decomposizione di $Z = z_1 \dots z_k$ per $P(i,j)$:

1) $z_k = x_i = y_j \implies z_1 \dots z_{k-1}$ è ammissibile per $P(i-1, j-1)$

2) $z_k \neq x_i \implies z_1 \dots z_k$ è ammissibile per $P(i-1, j)$

3) $z_k \neq y_j \implies z_1 \dots z_k$ è ammissibile per $P(i, j-1)$

Dim

1) Per assumzione $M_Z = (i_Z, j_Z) : (i_Z, j_Z)$ con

a) $1 \le i_Z[c] < i_Z[c+1] \le i_Z[c+1] \le n$

b) $1 \le j_Z[c] < j_Z[c+1] \le j_Z[c+1] \le m$

c) $z_c = x_{i_Z[c]} = y_{j_Z[c]} \quad \forall\, 1 \le c \le k$

Ma $i_Z[k] < i_Z[k+1] \le n$ e $j_Z[k] < j_Z[k+1] \le m$

per quanto due e due c) con $1 \le c \le k-1$

$1 \le i_Z[c] < i_Z[c+1] \le n-1$ e $1 \le j_Z[c] < j_Z[c+1] \le m-1$

da quest due e da c) con $1 \le c \le k-1$

otteniamo che $I = i_Z \dots i_{Z[k-1]}$ è ammissibile per $P(i-1, j-1)$.

2) Per assumzione $M_Z = (i_Z, j_Z) : (i_Z, j_Z)$ con

a) $1 \le i_Z[c] < i_Z[c+1] \le i_Z[c+1] \le n$

b) $1 \le j_Z[c] < j_Z[c+1] \le j_Z[c+1] \le m$

c) $z_c = x_{i_Z[c]} = y_{j_Z[c]} \quad \forall\, 1 \le c \le k$

Inoltre se $z_k \neq x_i \implies i_Z[k] < i$

Quindi $M_Z = (i_Z, j_Z) : \dots : (i_Z[k], j_Z[k])$ è t.c.

$1 \le i_Z[c] < i_Z[c+1] \le i_Z[k] < i \implies i_Z[c] < i$

$1 \le j_Z[c] < j_Z[c+1] \le j_Z[k] \le j$ (da b.)

c) è vera per assunzione.

$\implies Z$ è ammissibile per $P(i-1, j)$.

3) analogo a 2)

TEOREMA 2

1) Se $z_1 \dots z_k$ è sol ammissibile per $P(i-1, j-1)$

e $x_i = y_j$, allora $z_1 \dots z_k z_{k+1}$ (con $z_{k+1} = x_i = y_j$)

è ammissibile per $P(i,j)$

2) Se $z_1 \dots z_k$ è ammissibile per $P(i-1, j-1)$,

allora $z_1 \dots z_k$ è ammissibile per $P(i, j-1)$

3) Se $z_1 \dots z_k$ è ammissibile per $P(i-1, j)$,

allora $z_1 \dots z_k$ è ammissibile per $P(i, j)$

Dim

1) Se $Z = z_1 \dots z_k$ è ammissibile per $P(i-1, j-1)$

allora $M_Z = (i_Z, j_Z) : \dots : (i_{Z[k]}, j_{Z[k]})$ è t.c.

a) $1 \le i_Z[c] < i_Z[c+1] < i$

b) $1 \le j_Z[c] < j_Z[c+1] < j$

c) $z_c = x_{i_Z[c]} = y_{j_Z[c]} \quad \forall\, 1 \le c \le k$

Inoltre $x_i = y_j$, quindi posto

$z_{k+1} = x_i = y_j$, $i_{Z[k+1]} = i$ e $j_{Z[k+1]} = j$ otteniamo che

$Z' = z_1 \dots z_{k+1} z_{k+2}$

$M_{Z'} = M_Z \cup \{(i_{Z[k]}, j_{Z[k]}) = (i_Z[k], j_Z[k]), (i_{Z[k+1]}, j_{Z[k+1]})\}$

è t.c.a) $1 \le i_Z[k] < i_{Z[k+1]} \le i$

b) $1 \le j_Z[k] < j_{Z[k+1]} \le j$

c) $z_c = x_{i_Z[c]} = y_{j_Z[c]} \quad \forall\, 1 \le c \le k$

$\implies Z'$ è ammissibile per $P(i,j)$

2) e 3) sono molto semplici e lasciati come esercizio

Sottostruttura Ottima

TEOREMA 3

$Z = z_1 \dots z_k$ sol. ottima per $P(n,m)$, allora:

1) $x_n = y_m \implies z_k = x_n = y_m$ e $z_1 \dots z_{k-1}$ è ottima per $P(n-1, m-1)$ (2. b)

2) $x_n \neq y_m \implies$ 2.1) $z_k \neq x_n \implies z_1 \dots z_k$ è ottima per $P(n-1, m)$

2.2) $z_k \neq y_m \implies z_1 \dots z_k$ è ottima per $P(n, m-1)$

Dim

1. a) Per assurdo assumiamo che

(*) $z_k \neq x_n$ (e quindi $z_k \neq y_j$ poiché $x_n = y_j$).

$M_Z = (i_Z, j_Z) : \dots : (i_{Z[k]}, j_{Z[k]})$ è t.c.

a) $1 \le i_Z[c] < i_Z[c+1] < n$

b) $1 \le j_Z[c] < j_Z[c+1] < m$

c) $z_c = x_{i_Z[c]} = y_{j_Z[c]} \quad 1 \le c \le k$

Ma allora, per (*) e c), abbiamo che

$i_{Z[k]} \neq i \implies i_{Z[k]} < i$

$j_{Z[k]} \neq j \implies j_{Z[k]} < j$

Ma $x_i = y_j$, quindi posto

$z_{k+1} = x_i = y_j$

$i_{Z[k+1]} = i$

$j_{Z[k+1]} = j$

Possiamo costruire la sequenza

$Z' = z_1 \dots z_k z_{k+1}$

corrispondente al matching

$M_{Z'} = (i_Z, j_Z) : (i_{Z[k]}, j_{Z[k]}) : (i_{Z[k+1]}, j_{Z[k+1]})$

che soddisfa i vincoli

a) $1 \le i_Z[c] < \dots < i_{Z[k]} < i$

b) $1 \le j_Z[c] < \dots < j_{Z[k]} < j$

c) $z_c = x_{i_Z[c]} = y_{j_Z[c]} \quad \forall\, 1 \le c \le k+1$

Quindi Z' è ammissibile per $P(i,j)$ ed è più lunga di $Z \implies$ Z non ottima $\implies$ **Contraddizione!!**

1.b) Assumiamo $z_1 \dots z_{k-1}$ una ottima

Per $P(i-1, j-1)$, allora $\exists\, z^*$

$z^* = z_1^* \dots z_r^*$ ammissibile per $P(i-1, j-1)$

con $r > k-1$ e

$M_{z^*} = (i_1^*, j_1^*) \dots (i_r^*, j_r^*)$ t.c.

a) $1 \le i_1^* < \dots < i_r^* \le i-1$

b) $1 \le j_1^* < \dots < j_r^* \le j-1$

c) $z_\ell^* = x_{i_\ell} = y_{j_\ell}$ $\forall$ ℓ da 1 a r

Ma allora, posti:

$z_{r+1}^* = x_i = y_j$, $z_1^* \dots z_r^* z_{r+1}^*$, $i_{r+1}^* = i$, $j_{r+1}^* = j$

il matching $(i_1^*, j_1^*) \dots (i_r^*, j_r^*)(i_{r+1}^*, j_{r+1}^*)$

è ammissibile per $P(i,j)$

e ha lunghezza $r+1 > k$

$\Rightarrow$ z non è ottima $\Rightarrow$ Contraddizione!!

2) e 3) lasciata come esercizio

2) FUNZIONE DI COSTO OTTIMO

$$C(i,j) = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 \\ 1 + C(i-1, j-1) & \text{se } x_i = y_j \\ \max\{C(i-1, j),\ C(i, j-1)\} & \text{altrimenti} \end{cases}$$

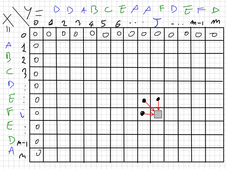
$P(n,m)$ richiede $n \cdot m$ sottoproblemi distinti.

Utilizzando matrice di dimensione $n \cdot m$ possiamo calcolare $C(i,j)$ in tempo costante e ottenere un algoritmo il costo complessivo $\Theta(n \cdot m)$

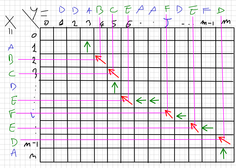
3) CALCOLO COSTO OTTIMO

X = A B C D E F E D A (1 2 3 4 5 6 7 8 9)

Y = D D A B C E A A F D E F D (1 2 3 4 5 6 7 8 9 10 11 12 13)



C[n,m]



S[n,m]

```
COSTO-OTTIMO (X[ ], Y[ ], n, m)
FOR j = 0 TO m DO
    C[0,j] = 0
FOR i = 1 TO n DO
    C[i,0] = 0
FOR i = 1 TO n DO
    FOR j = 1 TO m DO
        IF x[i] = y[j] THEN
            C[i,j] = 1 + C[i-1, j-1]
            S[i,j] = "↖"
        ELSE
            IF C[i-1,j] ≥ C[i,j-1] THEN
                C[i,j] = C[i-1,j]
                S[i,j] = "←"
            ELSE
                C[i,j] = C[i,j-1]
                S[i,j] = "↑"
```

4) RICOSTRUZIONE SOLUZIONE OTTIMA

```
PRINT_SOL (i, j, S[ ], X[ ])
    IF i > 0 ∧ j > 0 THEN
        IF S[i,j] = "↖" THEN
            PRINT_SOL (i-1, j-1, S, X)
            PRINT X[i]
        ELSE IF S[i,j] = "↑" THEN
            PRINT_SOL (i, j-1, S, X)
        ELSE
            PRINT_SOL (i-1, j, S, X)
```

ALBERO OTTIMO DI RICERCA

Date n chiavi $k_1, k_2, \dots k_n$ ordinate in modo non decrescente $k_1 \le k_2 \le \dots \le k_n$

e una distribuzione di probabilità sulle n chiavi:

p_i è la probabilità che la chiave k_i venga cercata.

Dato un albero binario di ricerca T caratterizzato da n chiavi, e detta $d_T(i)$ la profondità della chiave k_i in T,

$$C(T) = \sum_{i=1}^{n} \left(d_T(i)+1\right) * p_i$$ [Tempo medio di ricerca]

definito il costo medio di ricerca in T (assumendo che ogni ricerca abbia un esito positivo, cioè che la chiave cercata sia presente nell'albero)

Problema: Costruire l'albero binario di ricerca che minimizza il tempo medio di ricerca

Esempio:

i	1	2	3	4	5
p_i	0,1	0,2	0,2	0,15	0,3

T_1 [albero bilanciato]

$$C(T_1) = (0+1)\cdot 0{,}2 + (1+1)\cdot 0{,}1 + (2+1)\cdot 0{,}3 + (1+1)\cdot 0{,}15 + (2+1)\cdot 0{,}2 + (2+1)\cdot 0{,}3 = 2{,}1$$

T_2 [albero ottimo]

$$C(T_2) = (0+1)\cdot 0{,}3 + (1+1)\cdot 0{,}2 + (1+1)\cdot 0{,}1 + (2+1)\cdot 0{,}2 + (2+1)\cdot 0{,}15 + (3+1)\cdot 0{,}05 = 2{,}05$$

i	1	2	3	4	5
p_i	0,1	0,2	0,2	0,15	0,3

T_3

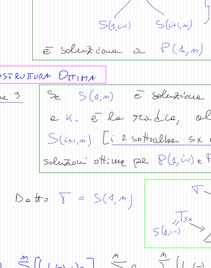
$$C(T_3) = (0+1)\cdot 0{,}3 + (1+1)\cdot 0{,}1 + (2+1)\cdot 0{,}3 + (2+1)\cdot 0{,}15 + (3+1)\cdot 0{,}2 = 2{,}15$$

[chiavi a maggior probabilità più in alto]

1) CARATTERIZZAZIONE SOLUZIONE OTTIMA

Ogni soluzione ammissibile deve essere un albero binario di ricerca che contiene tutte le chiavi $k_1, \dots k_n$

Tale albero avrà una radice k_r e forma:

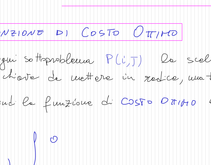


Teorema 1

Se $S(1,n)$ è soluzione al problema $P(1,n)$ e k_r è radice, allora $S(1, r-1)$ e $S(r+1, n)$ sono soluzioni ai problemi $P(1, r-1)$ e $P(r+1, n)$, rispettivamente

Teorema 2:

Se $S(1, i-1)$ e $S(i+1, n)$ sono sol per $P(1, i-1)$ e $P(i+1, n)$, rispettivamente, allora



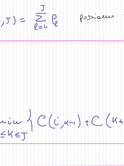
è soluzione a $P(1,n)$

SOTTOSTRUTTURA OTTIMA

Teorema 3

Se $S(1,n)$ è soluzione ottima per $P(1,n)$ e k_r è la radice, allora $S(1, r-1)$ e $S(r+1, n)$ [i sottoalberi sx e dx di k_r] sono soluzioni ottime per $P(1, r-1)$ e $P(r+1, n)$, rispettivamente

Dim Detto $T = S(1,n)$



$$C(T) = \sum_{j=1}^{n} \left[\left(d_T(j)+1\right) p_j\right] = \sum_{j=1}^{n} p_j + \sum_{j=1}^{n} \left(d_T(j)\cdot p_j\right) =$$

$$= \sum_{j=1}^{n} p_j + \sum_{j=1}^{i-1} \left(d_T(j)\cdot p_j\right) + d_T(i)\, p_i + \sum_{j=i+1}^{n} \left(d_T(j)\cdot p_j\right)$$

$$= \sum_{j=1}^{n} p_j + \sum_{j=1}^{i-1} \left[\left(d_{T_{sx}}(j)+1\right) p_j\right] + \sum_{j=i+1}^{n} \left[\left(d_{T_{dx}}(j)+1\right) p_j\right] \le$$

$$d_T(j) = d_{T_{sx}}(j) + 1 \quad \forall\, 1 \le j < i$$

$$d_T(j) = d_{T_{dx}}(j) + 1 \quad \forall\, i < j \le n$$

$$= \sum_{j=1}^{n} p_j + C(T_{sx}) + C(T_{dx}) = \sum_{j=1}^{n} p_j + C\left(S(1, i-1)\right) + C\left(S(i+1, n)\right)$$

$$C\left(S(1,n)\right) = \sum_{j=1}^{n} p_j + C\left(S(1, i-1)\right) + C\left(S(i+1, n)\right)$$

Il resto della dimostrazione segue il solito ragionamento per assurdo ed è lasciato come esercizio.

2) FUNZIONE DI COSTO OTTIMO

Per ogni sottoproblema $P(i,j)$ la scelta locale è la chiave da mettere in radice, usata tra $k_i, k_{i+1}, \dots, k_j$

Quindi la funzione di costo ottimo è:

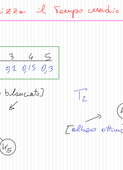
$$C(i,j) = \begin{cases} 0 & \text{se } i > j \\ \min\limits_{i \le k \le j} \left\{ \sum\limits_{p=i}^{j} p_p + C(i, k-1) + C(k+1, j) \right\} & \text{altrimenti} \end{cases}$$

(chiave da radice; $w(i,j)$ indipendente da k)

Ponendo, quindi, $w(i,j) = \sum_{p=i}^{j} p_p$ possiamo scrivere:

$$C(i,j) = \begin{cases} 0 & \text{se } i > j \\ w(i,j) + \min\limits_{i \le k \le j} \left\{ C(i, k-1) + C(k+1, j) \right\} & \text{altrimenti} \end{cases}$$

3) CALCOLO DEL COSTO OTTIMO




```
COSTO_OTTIMO (n, P[]) 
  FOR i = 1 TO n+1 DO
      C[i, i-1] = 0
  FOR i = n TO 1 DO
      w = 0
      FOR j = i TO n DO
          w = w + Pj
          C = 0
          FOR k = i TO j DO
              t = w + C[i, k-1] + C[k+1, j]
              IF t < C THEN
                  C = t
                  s = k
          C[i, j] = C
          S[i, j] = s
```

$\forall\, i \le j : w(i,j) = \sum_{k=i}^{j} p_k$ */

4) COSTRUZIONE ALBERO OTTIMO

Lasciato come esercizio

PROBLEMA DELLO ZAINO

Siano dati: n oggetti $O_1 O_2 \dots O_n$ ciascuno con un peso p_i e un valore v_i. Sia dato un contenitore (lo zaino) che può sostenere al massimo un peso W (capienza).
Calcolare l'insieme di oggetti di valore massimo che lo zaino è in grado di contenere.
(valore dello zaino è inteso come la somma dei valori degli oggetti in esso contenuti.)

Percorsi Minimi in Grafi Pesati: Algoritmo di Bellman-Ford

Dato un grafo $G=(V,E)$ e una funzione di peso $w: E \to \mathbb{R}$ a cui associa a ogni arco un peso (reale), il peso di un cammino (o percorso) è la somma dei pesi degli archi in π.

Nota: Se il peso contiene archi a pesi negativi, il problema non ammette soluzione.

Possiamo definire $P(s,t)$ come il problema di calcolare il percorso minimo da s a t, in G.

Una soluzione ammissibile per $P(s,t)$ è:
- Una sequenza $S = \langle s, v_1, \dots, v_k, t \rangle$ è ammissibile per $P(s,t)$ se:
  1) $s, v_1, \dots, v_k, t \in V$
  2) $(s, v_1), (v_1, v_2), \dots, (v_k, t) \in E$ $\forall$ archi

Ogni soluzione ammissibile per $P(s,t)$ (i.e., ogni percorso da s a t) è un percorso tra le soluzioni di Sottoproblemi.

1) $S(s,x) = \langle s, \dots, v_k, x \rangle$ Sì ammissibile per $P(s,x)$
$S(s,y) = \langle s, \dots, v_k, y \rangle$
$S(x,t) = \langle x, \dots, t \rangle$ } in cui $s \to x \to t$

Una soluzione ammissibile per $P(s,t)$ è $P(s,x)$, $P(x,t)$

(la dimostrazione è ovvia, perché ogni percorso è costituito da percorsi $\langle \dots \rangle$ intermedi tutti presi in un percorso π un percorso)

2) Se $S(s,x) = \langle s, \dots, v_k, x \rangle$ e $S(x,y) = \langle x, \dots, t \rangle$ sono ammissibili per $P(s,x)$ e $P(x,y)$, allora $S' = S(s,x) \cdot S(x,y) = \langle s, \dots, v_k, x, \dots, t \rangle$ è ammissibile per $P(s,y)$.

Anche questo è ovvio, poiché la concatenazione di 2 percorsi, tali che il primo termina nel vertice da cui inizia il secondo, è un percorso nel grafo.

Possiamo considerare anche la seguente variante di decomposizione in cui prendiamo sempre $y=t$:

$$S(s,x) = \langle s, \dots, v_k, x \rangle \Rightarrow (x,t) : S(x,t)$$
$$\Rightarrow \langle s, \dots, v_k, x, t \rangle$$
$$S(x,t)$$

La proprietà di decomposizione dello spazio soluzione assicura (soprattutto ammissibile) anche per questa decomposizione

Problema: La decomposizione data sopra non è sempre ben definita.

Es

$P(s,x) \Rightarrow (s,x) \cdot P(x,t) \Rightarrow (s,x) \cdot (x,s) \cdot P(s,x)$

La ricorsione non è ben definita e non garantisce terminazione perché per risolvere il problema $P(s,t)$ è necessario risolvere il problema $P(s,t)$ stesso: $P(s,t)$ dipende da se stesso.

Osservazione 1: Il problema sorge quando il grafo G in input contiene cicli. [perché?]

Osservazione 2: Un percorso minimo non deve visitare 2 vertici e un vertice intermedio semplice, non contiene sottopercorsi di vertici (un ciclo).

Dim: Per assurdo, assumiamo che il percorso minimo da v_0 a v_k contenga 2 occorrenze dello stesso vertice v:

$$v_0 \to \dots \to v \to \dots \to v \to \dots \to v_k$$

$$w(\pi) = \sum_{i=1}^{k} w(v_{i-1}, v_i) = \sum_{i=1}^{j-1} w(v_{i-1}, v_i) + \sum_{i=j}^{k} w(v_{i-1}, v_i)$$

sottopercorso da v_0 a v — sottopercorso da v a v_k

Abbiamo 3 casi:

1) Il peso dal sottopercorso π'' è negativo

$$w(\pi'') = \sum_{i=j}^{k} w(v_{i-1}, v_i) < 0$$

In questo caso un cammino minimo da v_0 a v_k [perché?]

2) Il peso dal sottopercorso π'' è un negativo

$$w(\pi'') = \sum_{i=j}^{k} w(v_{i-1}, v_i) > 0$$

In questo caso il percorso $\pi' \cdot \pi''$, ottenuto eliminando il sottopercorso π'' da π, ha peso $\leq w$. Quindi se π è ottimo, lo è anche π'.

Osservazione 2: è permesso di considerare che un percorso minimo tra 2 vertici di un grafo $G=(V,E)$ ha al massimo lunghezza $|V|-1$ archi.

1) Caratterizzazione Struttura Soluzione Ottima

Possiamo allora definire il problema in questo modo:

$P(s,x,|V|-1)$: Problema di trovare il percorso minimo da s a x contenente al più $|V|-1$ archi.

La decomposizione risultata quindi (quando $|V|=n$)

Sia $S(s,x,n-1) = \langle s, \dots, v_k, x \rangle$ soluzione a $P(s,x,n-1)$:

caso 1) Se $v_k = s$ allora:

$$S(s,x,n-1) = \langle s, \dots, s \rangle \quad \text{con } s = v_k$$
$$\langle s, \dots, v_{k-1} \rangle = S(s,s,n-2)$$

caso 2) Se $v_k \neq s$ allora

$\langle s, \dots, v_k \rangle$ soluzione a $P(s,x,n-1)$

Notate che al 2° sottoproblema è un caso passando da problema a sottoproblema, non posso essere usato dentro un parametro fisso completo.

caso 1) $S(s,x,n-1) = (s,s) \cdot S(s,x,n-2)$ con $s,x \in V$

caso 2) $S(s,x,n-1) = S(s,x,n-1)$

Sottostruttura Ottima

Se $S(s,x,n-1) = \langle s, \dots, v_k, x \rangle$ è percorso ottimo (il peso minimo) per il problema $P(s,x,n-1)$ allora

1) Se $v_k = s$ allora $S(s,x,n-1) = \langle s, \dots, s \rangle$ è ottimo per $P(x,n-2)$

2) Se $v_k \neq s$ allora $S(s,x,n-1) = \langle s, \dots, v_k \rangle$ è ottimo per $P(s,n-1)$

Dim:
1) Per assurdo, se $\langle s, \dots, s \rangle$ non è ottimo per $P(s,x,n-1)$, allora è un percorso $S' = \langle s, \dots, v_k' \rangle$ ammissibile per $P(s,x,n-1)$ [perché contenente al massimo $n-2$ archi.]
Tale che $w(S') < w(S(s,x,n))$
Ma $S'(s,x)$ è ammissibile per $P(s,n-1)$ e $w(S') = w(s,s) + w(S') < w(s,s) + w(S(s,x,n-1)) = w(S(s,n-1))$
$\Rightarrow S(s,n-1)$ non è ottimo $\Rightarrow$ assurdo.

2) Banale e lasciato come esercizio.

2) Funzione di Costo Ottimo

$$C(s,i) = \begin{cases} 0 & \text{se } i=s \text{ e } s \\ \infty & \text{se } i \neq s \text{ e } i=s \\ \min\{C(s,j) + w(j,s) : C(s,j) < \infty\} & \text{altrimenti} \end{cases}$$

Il numero di sottoproblemi distinti è $|V| \cdot |V| = n^2$

Infatti dopo la scelta ottima di ogni sottoproblema (i.e., il vertice v_k immediato di costo) rimane ad più $|V|$ problemi, un caso complessivo è $O(n^2)$

3) Calcolo del Costo Ottimo

Come negli esempi precedenti, usiamo una tabella $C[i,j]$ di dimensione $|V| \times |V|$, poiché il primo parametro i varia nell'insieme $\{1, \dots, |V|\}$ e il secondo parametro j nell'insieme $\{1, \dots, |V|\}$. Il costruire di ogni cella $C[i,j]$ dipende solo dal valore di celle che stanno nella colonna precedente $j-1$ di cui la cella di coordinate $C[v,j-1]$ con $v \in Adj[i]$:

```
Costo-Ruino (G, s)
  n = |V|
  For each v ∈ V\{s} do
      C[v,0] = ∞
  C[s,0] = 0
  For i = 1 to n-1 do
      For each v ∈ V do
          C[v,i] = C[v,i-1]
          S'[v,i] = ∞
          For each x ∈ Adj[v] do
              If C[v,i] < d(s,v) + C[x,i-1] Then
                  C[v,i] = w(v,x) + C[x,i-1]
                  S'[v,i] = x
```

E' immediato osservare che il tempo richiesto è $O(n^3)$. Si può dimostrare che, in effetti, il tempo richiesto è $\Theta(|V| \cdot |E|)$ [Perché?]

Richiede spazio $\Theta(|V|^2)$, un lato esiste una dimostrazione che usa solo spazio $\Theta(|V|)$.

4) Costruire Percorso Minimo

- Se, al termine, troviamo che $C[v,|V|-1] = \infty$, allora un t un s irraggiungibile da s
- Se $C[v,|V|-1] \neq \infty$, allora esiste un percorso da s a t. Il percorso minimo può essere ottenuto ricostruito ricorsivamente dal seguente algoritmo.

```
Percorso-Minimo (G, t, v, i, S())
  If t ≠ v Then
      x = S'[v,i]
      If x ≠ NIL Then
          Print x
          Percorso-Minimo (G, t, x, i-1, S())
      Else
          Percorso-Minimo (G, t, v, i-1, S())
  Else Print v
```

Costo : $O(|V|)$

ALGORITMI GREEDY

PROGRAMMAZIONE DINAMICA valuta tutte le possibili scelte locali, calcolando per ciascuna il valore ottimo complessivo.
Il costo complessivo associato a una scelta è calcolato sommando il contributo locale di quella scelta con il costo ottimo dei sottoproblemi risultanti dalla scelta stessa.
Tra queste scelte selezionare, poi, quella che fornisce il costo complessivo migliore.

Un ALGORITMO GREEDY si basa sull'idea di risolvere il problema costruendo la soluzione in modo incrementale.
Per ogni sottoproblema si considera sempre una sola scelta localmente ottima.

La valutazione della bontà della scelta è fatta unicamente considerando il sottoproblema corrente e le possibili scelte, quindi basandosi solo su informazioni locali.

Limitarsi a valutazioni locali della scelta tipicamente porta ad algoritmi più efficienti.

Non sempre la soluzione che si ottiene è ottima.

I problemi che ammettono soluzioni ottime tramite tecniche greedy soddisfano due condizioni:

1) SOTTOSTRUTTURA OTTIMA
2) PROPRIETÀ SCELTA GREEDY

La condizione 1) è la stessa richiesta per l'applicazione della PROGRAMMAZIONE DINAMICA.

La condizione 2), invece, richiede che la scelta greedy, che viene effettuata per ogni sottoproblema, faccia necessariamente parte di una soluzione ottima a quel sottoproblema.

SCHEDULAZIONE INTERVALLI NON PESATI

Input: Un insieme di attività (task) $T = \{t_1, \dots, t_n\}$
Ogni Task i ha tempo inizio $S(i)$ e Tempo fine $F(i)$
$[S(i), F(i))$ intervallo di i

Output: Insieme di Task di dimensione massima tale che nessun Task si sovrapponga agli altri

Nota: Qui la condizione di ottimalità è determinata solo dal numero di intervalli (Task) schedulati, non dal loro valore complessivo.
Questo rende il problema più semplice da risolvere

Come per la versione pesata del problema, assumiamo che gli intervalli siano già ordinati per Tempo di terminazione $F(i)$ crescente:

$T = t_1, t_2 \dots t_n$ con $f(i_j) \le f(i_{j+1})$ $\forall$ indici

Useremo la seguente notazione di problema e decomposizione

$P(i)$: Problema di schedulare gli intervalli (task) da t_1 a t_m che sono compatibili con i $i \ge 0$

Assumiamo un task fittizio 0 con $S[0]=F[0]=0$, il problema principale è, quindi $P(0)$.

DECOMPOSIZIONE:

$S(0) = J_a\ J_j \dots J_m$ ammissibile per $P(0)$
- $0 < J_a < J_2 < \dots < J_m \le n$
- $\forall$ indice $J_k \cap J_{k+1} = \emptyset$ [J_k e J_{k+1} sono compatibili]
- $0 \cap J_1 = \emptyset$ [0 e J_1 sono compatibili]

Nota: poiché i Task sono ordinati, se J_k è compatibile con J_{k+1} $[J_k \cap J_{k+1} = \emptyset]$ per ogni $1 \le j < k$, allora J_k è compatibile con tutti i J_m tali che $m > k$.

$\left[\begin{array}{l} J_k \cap J_{k+1} = \emptyset \Rightarrow f[J_k] < S[J_{k+1}] \\ J_{k+1} \cap J_{k+2} = \emptyset \Rightarrow f[J_{k+1}] < S[J_{k+2}] \\ \text{con } S[J_{k+1}] \le f[J_{k+1}] \Rightarrow f[J_k] < S[J_{k+2}] \\ \text{(per transitività)} \end{array} \right]$

e quindi $J_k \cap J_{k+2} = \emptyset$

Iterando il ragionamento si ottiene ex Task:

In generale

$S(i) = J_a \dots J_m$ ammissibile per $P(i)$
se
- $i < J_a < \dots < J_m \le n$
- $\forall$ indice $J_k \cap J_{k+1} = \emptyset$
- $i \cap J_a = \emptyset$

1.- Se $J_a\ J_3 \dots J_m$ è ammissibile per $P(i)$
allora $J_3 \dots J_m$ è ammissibile per $P(J_a)$

2.- Se $J_a\ J_3 \dots J_m$ è ammissibile per $P(i)$
e $J_3'\ J_3' \dots J_p'$ è ammissibile per $P(J_a)$
allora $J_3\ J_3' \dots J_p'$ è ammissibile per $P(i)$

3.- Se $J_a\ J_3 \dots J_m$ è ottima per $P(i)$
allora $J_3 \dots J_m$ è ottima per $P(J_a)$

Dimostrare le 3 proprietà di decomposizione sopra è lasciato come esercizio.

La decomposizione appena definita e il fatto che il costo di una soluzione è dato dal n° di intervalli (task) che esse contiene, suggeriscono una funzione di costo ottimo definita come

$$C(i) = \begin{cases} 0 & \text{se } \nexists \text{ task } j \text{ con } f[j] \ge s[i] \\ 1 + \max\limits_{\substack{j \\ f[j] \ge s[i]}} \{ C(j) \} & \text{altrim.} \end{cases}$$

Per ogni problema $P(i)$, vengono Tentate Tutte le scelte $i < j < m$ che sono compatibili con i e, per ciascuna di esse, viene risolto il sottoproblema $P(j)$ corrispondente

È possibile evitare di valutare l'effetto di ogni possibile scelta (cioè risolvere Tutti i sottoproblemi $P(j)$) e ottenere la scelta ottima per $P(i)$ confrontando tra di loro solo la possibile scelta?

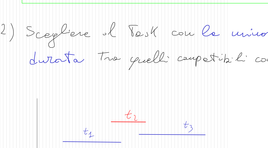
UNA SOLUZIONE GREEDY

Un *Algoritmo Greedy* utilizza una politica di scelta locale

Per ogni sottoproblema, effettua una sola scelta, quella che sembra la migliore, senza calcolare il costo ottimo complessivo influito da ciascuna scelta.

Possibili esempi:

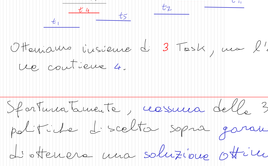
1) Scegliere sempre il Task che inizia prima tra quelli compatibili con i.



Otteniamo insieme con 1 Task, ma l'ottimo ne contiene 4.

Non viene conto del fatto che un Task può venire scartato la risorsa per lungo tempo

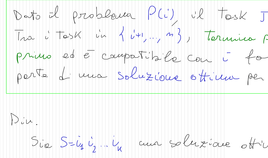
2) Scegliere il Task con la minor durata tra quelli compatibili con i.



Otteniamo insieme con 1 task, ma l'ottimo ne contiene 2.

Non viene conto che un Task potrebbe essere incompatibile con molti Task

3) Scegliere il Task, tra quelli compatibili con i, compatibile col maggior n° di Task disponibili.



Otteniamo insieme di 3 Task, ma l'ottimo ne contiene 4.

Sfortunatamente, nessuna delle 3 politiche di scelta sopra garantisce di ottenere una soluzione ottima.

POLITICA GREEDY OTTIMA PER P(i)

Scegliere il Task, tra quelli compatibili con i, che Termina per primo.

Questa politica garantisce di ottenere sempre una soluzione ottima.

PROPRIETÀ DI SCELTA GREEDY

Dato il problema P(i), il Task J che, tra i Task in $\{i_1, i_2, \dots, i_n\}$, Termina per primo ed è compatibile con i fa parte di una soluzione ottima per P(i).

Dim.

Sia $S = i_{j_1} i_{j_2} \dots i_{j_k}$ una soluzione ottima per P(i). Poiché S è ordinata per tempi di terminazione crescenti, i_{j_1} è il Task che termina per primo tra quelli in S, cioè

$$f_{j_1} \leq f_{j_2} \quad \text{per ogni } z \in \{1 \dots k\}$$

Sia J il Task in $\{i_1, i_2, \dots, i_n\}$ che Termina per primo ed è compatibile col Task i.

Ovviamente $J \in i_{j_z}$ [Perché?]

Abbiamo 2 casi:

1) $J = i_{j_1}$, allora la tesi è verificata banalmente

2) $J \neq i_{j_2}$. Ma $J \in i_{j_2} \Rightarrow J < i_{j_2}$. Poiché i Task sono ordinati, vale $f[J] \leq f[i_{j_2}]$

Inoltre J è compatibile con i, quindi $f[J] < S[j_2]$

Sia $S^* = J\, i_{j_2} \dots i_{j_k}$.

S^* è ammissibile per P(i).

Infatti:

- $i < J < i_{j_2} < i_{j_3} < \dots < i_{j_k} \leq n$
- $J \cap i_{j_z} = \emptyset$ e $i_{j_z} \cap i_{j_m} = \emptyset \quad \forall z, m \in \{2 \dots k\}$
- $i \cap J = \emptyset$ [per scelta di J]

Ma il n° di Task in S^* è lo stesso che in S $\Rightarrow S^*$ è anch'essa ottima $\Rightarrow$ tesi ■

ALGORITMO GREEDY

Il seguente algoritmo ottiene la soluzione ottima, facendo sempre la scelta greedy e sfruttando il fatto che i Task sono ordinati per Tempo di Terminazione (f[i]) crescente.

```
SCHEDULER (T[1..n])
    SOL = ∅
    t = 0
    FOR i = 1 TO n DO
        IF S[i] > t THEN
            SOL = SOL ∪ {i}
            t = f[i]
    RETURN SOL
```

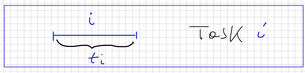
Tempo di esecuzione $\Theta(n)$, se l'operazione di unione (SOL ∪ {i}) è implementata a Tempo costante (e.g. inserimento in Testa alla lista SOL).

$[\Theta(n \log n)$ se inizialmente ordinamento di T$]$

Scheduling con Minimo Tempo Medio di Completamento

Input: Un insieme di attività (task) $T = \{1, \dots, n\}$

Ogni Task i richiede tempo t_i per essere elaborato.
(t_i è la durata del Task i)



Output: Schedulazione di tutti gli n task che minimizza il Tempo medio di completamento degli n Task.

Sottostruttura Ottima

Consideriamo il problema $P(T, 0)$ di schedulare i Task in T a partire dal tempo 0 (zero).

$S = (s_1, i_1)\ (s_2, i_2)\ \dots\ (s_n, i_n)$ è ammissibile per $P(T, 0)$

(s_1: Tempo inizio Task i_1 ; s_n: Tempo inizio Task i_n)

se

1) $0 \le s_1$
2) $s_\ell + t_{i_\ell} \le s_{\ell+1}$ ✓ $1 \le \ell \le n-1$
3) $\{i_1, i_2, \dots, i_n\} = T$ (S contiene tutti e soli i Task in T)

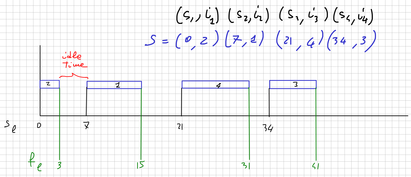
Costo di una soluzione $S = (s_1, i_1)\ (s_2, i_2)\ \dots\ (s_n, i_n)$

$$C(S) = \frac{1}{n} \sum_{\ell=1}^{n} f(\ell) \qquad \text{dove} \quad f(\ell) = s_\ell + t_{i_\ell}$$

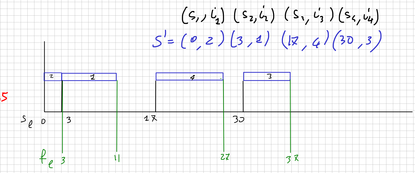
($f(\ell)$: Tempo di Completamento dell'ℓ-esimo Task in S (Task i_ℓ))

Osservazione: Possiamo ignorare soluzioni con intervalli di inattività (idle Time).

Task	durata
1	$(t_1 = 8)$
2	$(t_2 = 2)$
3	$(t_3 = 2)$
4	$(t_4 = 10)$



$C(S) = \frac{1}{4} \sum_{\ell=1}^{4} f_\ell = \frac{1}{4}(3 + 15 + 31 + 41) = 22{,}5$



$C(S') = \frac{1}{4} \sum_{\ell=1}^{4} f_\ell = \frac{1}{4}(3 + 11 + 27 + 33) = 43{,}5$

Quindi, una soluzione ottima non può ammettere idle Time

Possiamo restringerci a considerare solo soluzioni senza idle Time

$S = (s_1, i_1)\ (s_2, i_2)\ \dots\ (s_n, i_n)$ è ammissibile per $P(T, 0)$

se

1) $0 = s_1$
2) $s_\ell + t_{i_\ell} = s_{\ell+1}$ ✓ $1 \le \ell \le n-1$ (che implica $s_\ell = \sum_{h=1}^{\ell-1} t_{i_h}$)
3) $\{i_1, i_2, \dots, i_n\} = T$ (S contiene tutti e soli i Task in T)

Cioè, i tempi di inizio dei Task nella soluzione sono univocamente determinati dalla posizione dei Task in esse.

Possiamo quindi semplificare la formalizzazione come segue.

Formalizzazione del Problema

$S = i_1\, i_2 \dots i_n$ ammissibile per $P(T)$

se $\{i_1, i_2, \dots, i_n\} = T$

Costo di S

$$C(S) = \frac{1}{n} \sum_{\ell=1}^{n} f(\ell) \qquad \text{con} \quad f(\ell) = \sum_{k=1}^{\ell} t_{i_k}$$

Proprietà di Decomposizione e Sottostruttura Ottima

1) Se $S = i_1\, i_2 \dots i_n$ è ammissibile per $P(T)$, allora $S' = i_2 \dots i_n$ è ammissibile per $P(T \setminus \{i_1\})$

2) Se $S' = i_2' i_3' \dots i_k'$ è ammissibile per $P(T \setminus \{i\})$, allora $S = i \cdot S'$ è ammissibile per $P(T)$

3) Se $S = i_1\, i_2 \dots i_n$ è ottima per $P(T)$, allora $S' = i_2 \dots i_n$ è ottima per $P(T \setminus \{i_1\})$

3) Se $S = i_1 i_2 \dots i_n$ è ottima per $P(T)$, allora $S' = i_2 \dots i_n$ è ottima per $P(T \setminus \{t_{i_1}\})$

→ Dim.

$$C(S') = \frac{1}{n-1} \sum_{\ell=2}^{n} f'(i_\ell) = \frac{1}{n-1} \sum_{\ell=2}^{n} \left(\sum_{j=2}^{\ell} t_{i_j} \right)$$

$$C(S) = \frac{1}{n} \sum_{\ell=1}^{n} f(i_\ell) = \frac{1}{n} \cdot \sum_{\ell=1}^{n} \left(\sum_{j=1}^{\ell} t_{i_j} \right) =$$

$$= \frac{1}{n} \left[\sum_{\ell=1}^{n} t_{i_1} + \sum_{\ell=1}^{n} \left(\sum_{j=2}^{\ell} t_{i_j} \right) \right] = \quad (\text{perché?})$$

$$= \frac{1}{n} \left[\sum_{\ell=1}^{n} t_{i_1} + \sum_{\ell=2}^{n} \left(\sum_{j=2}^{\ell} t_{i_j} \right) \right] = \quad (\text{perché?})$$

$$= \frac{1}{n} \left[\sum_{\ell=1}^{n} t_{i_1} + (n-1) \cdot C(S') \right]$$

Assumendo ora S ottima per $P(T)$ ma S' non ottima per $P(T \setminus \{t_{i_1}\})$, avremmo che

$\exists\, S'' = i_2' i_3' \dots i_n'$ t.c. $C(S') > C(S'')$

Ma, per proprietà 2) di decomposizione, $S^* = i_1 \circ S'' = i_1 i_2' \dots i_n'$ è ammissibile per $P(T)$ e

$$C(S^*) = \frac{1}{n} \left[\sum_{\ell=1}^{n} t_{i_1} + (n-1) \cdot C(S'') \right]$$
$$C(S) = \frac{1}{n} \left[\sum_{\ell=1}^{n} t_{i_1} + (n-1) \cdot C(S') \right]$$

$\Rightarrow C(S^*) < C(S)$. Assurdo!!

SCELTA GREEDY

La politica di scelta che garantisce di ottenere sempre una soluzione ottima è la seguente

Schedulare per primo il Task con la minima durata tra quelli disponibili.

PROPRIETÀ DELLA SCELTA GREEDY

Dato il problema $P(T)$, con $|T| = n$.
Sia $i \in T$ il Task con la minima durata tra i Task in T.
Allora esiste una soluzione ottima $S^* = i_1 i_2 \dots i_n$ t.c. $i_1 = i$

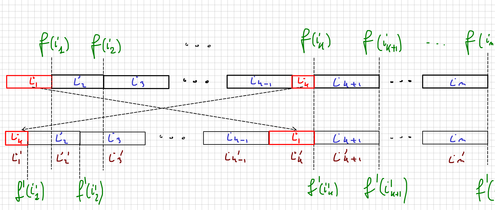
Dim.

Consideriamo una soluzione ottima $S = i_1' i_2' \dots i_n'$ per il problema $P(T)$.

Abbiamo 2 possibili casi:

1) $i_1 = i$: in tal caso la tesi vale banalmente

2) $i_1 \neq i$: una $i \in S$, poiché $i = i_k$ per qualche $2 \le k \le n$.
Inoltre, $t_i \le t_{i_1}$ poiché $i_1 \in T$ e i è il Task con durata minima in T.
Scambiando le posizioni di i_1 e i_k in S, otteniamo una nuova soluzione ammissibile S':



Cioè S' è definibile a partire da S come segue:

$$S' = i_1' i_2' \dots i_n'$$

con $i_1' = i_1$, $i_k' = i_1$ e $i_\ell' = i_\ell$ per ogni $\ell \neq 1$ e $\ell \neq k$

Vogliamo ottenere le seguenti relazioni tra S e S'

a) $\forall\, k \le \ell \le n$ $\quad \sum_{j=1}^{\ell} t_{i'_j} = \sum_{j=1}^{\ell} t_{i_j}$ $\quad [\, f'(i'_\ell) = f(i_\ell)\,]$

Il tempo di completamento dei task dal k-esimo all'n-esimo è lo stesso in S e S'.

Infatti, poiché $i_1 = i'_k$ e $i_k = i'_1$, abbiamo che:

$$f(i'_\ell) = \sum_{j=1}^{\ell} t_{i_j} = t_{i_1} + t_{i_2} + \cdots + t_{i_k} + \cdots + t_{i_\ell} =$$
$$= t_{i'_k} + t_{i'_2} + \cdots + t_{i'_1} + \cdots + t_{i'_\ell} = \sum_{j=1}^{\ell} t_{i'_j} = f'(i'_\ell)$$

b) $\forall\, 1 \le \ell \le k-1$ $\quad \sum_{j=1}^{\ell} t_{i'_j} \le \sum_{j=1}^{\ell} t_{i_j}$ $\quad [\, f'(i'_\ell) \le f(i_\ell)\,]$

Infatti, poiché $i'_1 = i_k$, abbiamo che:

$$f(i'_\ell) = \sum_{j=1}^{\ell} t_{i_j} = t_{i_1} + t_{i_2} + \cdots + t_{i_\ell}$$
$$\qquad\qquad\quad \| \quad \| \quad \|$$
$$f'(i'_\ell) = \sum_{j=1}^{\ell} t_{i'_j} = t_{i'_2} + t_{i'_3} + \cdots + t_{i'_\ell} = t_{i_k} + t_{i_2} + \cdots + t_{i_\ell}$$

Ma, per assunzione, $t_{i_k} \le t_{i_1}$, quindi

$$f'(i'_\ell) = \sum_{j=1}^{\ell} t_{i'_j} \le \sum_{j=1}^{\ell} t_{i_j} = f(i'_\ell)$$

Da a) e b) segue immediatamente che

$$C(S') \le C(S)$$

e, quindi, che S' è ottima se S lo è. ∎

ALGORITMO GREEDY DI SOLUZIONE

La proprietà di scelta greedy ci assicura che una soluzione ottima si ottiene calcolando una permutazione degli n task che ordini i Task in maniera crescente (non decrescente).

Il problema si riduce, quindi, al problema dell'ordinamento e il costo in tempo è

$$\Theta(n \log n).$$

PROBLEMA DELLO ZAINO FRAZIONARIO

Siano dati: n oggetti $\{O_1, O_2, \dots O_n\}$. Dell'oggetto O_i se ha a disposizione una quantità complessiva di peso P_i. La quantità totale P_i dell'oggetto O_i ha un valore pari a V_i. Sia dato un contenitore (lo zaino) che può sostenere al massimo un peso W (capienza).
Uno riempimento dello zaino indica quale porzione di ciascun oggetto inserire nello zaino, senza eccedere la sua capacità.
Calcolare il riempimento di valore massimo per lo zaino.
(valore dello zaino è inteso come la somma dei valori delle porzioni degli oggetti in esso contenute)

Decomposizione:
$P(O, w)$ → capacità residua dello zaino $(w \le W)$
→ insieme di oggetti a disposizione $(O \subseteq O)$

$S = (i_1, y_1) \dots (i_k, y_k)$ è ammissibile per $P(O, w)$ se
- $i_1, \dots, i_k \in O$
- $y_\ell \le P_{i_\ell} \quad \forall\, 1 \le \ell \le k$
- $\sum_{\ell=1}^{k} y_\ell \le w$

$$C(S) = \sum_{\ell=1}^{k} \frac{y_\ell}{P_{i_\ell}} \cdot v_{i_\ell}$$

$\frac{y_\ell}{P_{i_\ell}}$ è la frazione di peso dell'oggetto O_{i_ℓ} contenuta in S

$$C(S) = \sum_{\ell=1}^{k} y_\ell \cdot \frac{v_{i_\ell}}{P_{i_\ell}}$$

$\frac{v_{i_\ell}}{P_{i_\ell}}$ è il valore di una unità di peso dell'oggetto O_{i_ℓ}

SOTTOSTRUTTURA OTTIMA

Se $(i_1, y_1) \dots (i_k, y_k)$ è sol ottima per $P(O, w)$
allora $(i_2, y_2) \dots (i_k, y_k)$ è sol ottima per $P(O \setminus \{i_1\}, w - y_1)$

SCELTA GREEDY

Per ogni sottoproblema $P(O, w)$ scegliere (i_1, y_1) tale che:
1) i_1 è l'oggetto col massimo valore per unità di peso tra gli oggetti di O
i_1 è t.c. $\frac{v_{i_1}}{P_{i_1}} \ge \frac{v_i}{P_i} \quad \forall\, i \in O$
2) y_1 è la massima quantità di i_1 che può essere inserita nello zaino
$y_1 = \min(P_{i_1}, w)$

PROPRIETÀ DI SCELTA GREEDY

Una soluzione ottima al problema $P(O, w)$ contiene la scelta greedy

Dim. Sia $S = (i_1, y_1) \dots (i_k, y_k)$ una soluzione ottima per $P(O, w)$ e sia J l'oggetto col massimo valore per unità di peso. Cioè:
$$\frac{v_J}{P_J} \ge \frac{v_i}{P_i} \quad \forall\, i \in O$$
Abbiamo 2 casi:
a) J è contenuto in S.
Quindi esiste un $1 \le \ell \le k$ tale che $(i_\ell, y_\ell) \in S$ e $i_\ell = J$
Sappiamo che $y_\ell \le P_\ell$ per ammissibilità di S.
Abbiamo altri 2 sotto-casi:
1) $y_\ell = P_\ell$ o $y_\ell = w$.
In entrambi i casi (i_ℓ, y_ℓ) è la scelta greedy e la tesi è dimostrata
2) $y_\ell < P_\ell$ e $y_\ell < w$, allora prendiamo un qualsiasi indice $\pi \ne \ell$, con $1 \le \pi \le k$.
Sia (i_π, y_π) la π-esima scelta in S
e poniamo $x = \min(x_\pi, P_J - y_\ell)$.
esiste per forza se S è ottima e $y_\ell < w$
Se sostituiamo in S
(J, y_J) con $(J, y_J + x)$
(i_π, y_π) con $(i_\pi, y_\pi - x)$
otteniamo una nuova soluzione S' con lo stesso peso di S.
Il costo di S' è:
$$C(S') = C(S) - \frac{v_\pi}{P_\pi} \cdot x + \frac{v_J}{P_J} \cdot x$$
Ma $\frac{v_\pi}{P_\pi} \le \frac{v_J}{P_J} \Rightarrow C(S') \ge C(S)$.
Se $(J, y_J + x)$ non è ancora la scelta ottima (cioè $x < y_J + x < P_J$ e $y_J + x < w$) ripetiamo lo stesso ragionamento con un altro oggetto in S, fino a ottenere una soluzione con la scelta greedy $\Rightarrow$ Tesi
b) J non è contenuto in S.
Sia $1 \le \pi \le k$ e (i_π, y_π) la π-esima scelta in S e $x = \min(y_\pi, P_J)$
Sostituendo in S
(i_π, y_π) con $(i_\pi, y_\pi - x)$ [$x - y_\pi > 0$]
e inserendo
(J, x)
otteniamo una soluzione ammissibile (il peso è lo stesso) e il costo è tale che
$C(S') \ge C(S)$
per lo stesso ragionamento usato nel caso 1.b).
Se (J, x) non è la scelta greedy (cioè se $x < w$ e $x < P_J$) possiamo applicare il caso 1.b) ripetutamente fino a ottenerla.
La tesi vale quindi anche in questo caso.

OSSERVAZIONE: La strategia greedy non garantisce l'ottimo per il problema originario dello ZAINO 0-1.

CONTROESEMPIO
$W = 4 \qquad O = \{1, 2, 3\}$
$P_1 = 3 \qquad v_1 = 5$
$P_2 = P_3 = 2 \qquad v_2 = v_3 = 3$
$\frac{v_1}{P_1} = \frac{5}{3} \qquad \frac{v_2}{P_2} = \frac{v_3}{P_3} = \frac{3}{2} \qquad \left[\frac{5}{3} > \frac{3}{2}\right]$
SOLUZIONE GREEDY: $S = \{1\}$ $C(S) = 5$
SOLUZIONE OTTIMA: $S^* = \{2, 3\}$ $C(S^*) = 6$

PROBLEMI INTRATTABILI: LA CLASSE NP

Un problema X si dice **TRATTABILE** se esiste un algoritmo che risolve ogni sua istanza $x \in X$ in tempo **POLINOMIALE sulla dimensione di x**, cioè in tempo $O(n^k)$, dove $n = |x|$ e $k \geq 0$ è una costante positiva.

Un problema X che non è trattabile è detto **INTRATTABILE**.

La **classe dei problemi trattabili**, risolvibili in tempo polinomiale, è indicata con P.

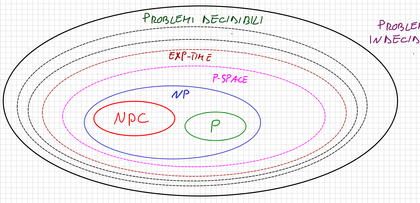
Un problema $X \in P$ se esiste un algoritmo che lo risolve in tempo polinomiale sulla sua dimensione.

Esistono molti problemi per cui è stata stabilita (dimostrata) la **non-trattabilità**, ma anche molti problemi per cui semplicemente non si è **mai trovata una soluzione in tempo polinomiale**.

La classe di questi ultimi problemi è indicata con **NPC** (problemi NP-COMPLETI).

Trovare una soluzione efficiente (in tempo polinomiale) per un qualsiasi problema in NPC significa poter risolvere **ogni problema** in NPC in tempo polinomiale.

Esiste una **gerarchia di classi di problemi**, che permette di classificare i problemi decidibili in base alla loro **difficoltà**.



PROBLEMI INDECIDIBILI (formulati da una macchina di Turing)

PROBLEMI DECIDIBILI:

- **EXP-TIME** = TEMPO ESPONENZIALE (scacchi, dama, ...)
- **P-SPACE** = SPAZIO POLINOMIALE (this, planning, ...)
- **NP** = TEMPO POLINOMIALE NON DETERMINISTICO (percorso massimo in un grafo, ...)
- **P** = TEMPO POLINOMIALE DETERMINISTICO (percorso minimo in un grafo, ...)

$$P \subseteq NP \subseteq P\text{-SPACE} \subseteq EXP\text{-TIME}$$

$$P \subset EXP\text{-TIME}$$

PROBLEMI DI DECISIONE E PROBLEMI DI OTTIMIZZAZIONE

La Teoria della complessità si concentra sullo studio della difficoltà dei problemi di decisione.

PROBLEMA DI DECISIONE: Problema la cui soluzione è SI/NO.

ESEMPIO

$\text{PATH}^{\text{É}}(G, v, u, k)$: Dato un grafo $G = \langle V, E \rangle$, due vertici v e u e un intero k, esiste un percorso da v a u in G con al più k archi?

Per ogni problema di ottimizzazione esiste un problema di decisione corrispondente.

ESEMPIO

$\text{MIN-PATH}(G, v, u)$: Dato un grafo $G = \langle V, E \rangle$ e due vertici v e u calcolare la lunghezza del percorso minimo da v a u in G.

Il problema $\text{PATH}^{\text{É}}(G, v, u, k)$ è il problema di decisione corrispondente a $\text{MIN-PATH}(G, v, u)$ nel seguente senso.
Se abbiamo un algoritmo per risolvere il problema $\text{MIN-PATH}(G, v, u)$, allora possiamo ottenere un algoritmo per risolvere $\text{PATH}^{\text{É}}(G, v, u, k)$:

> Basta risolvere $\text{MIN-PATH}(G, v, u)$. Se il risultato ottenuto è $\leq k$ rispondiamo SI, altrimenti rispondiamo NO.

Intuitivamente, $\text{PATH}^{\text{É}}(G, v, u, k)$ non è più difficile da risolvere di $\text{MIN-PATH}(G, v, u)$.
Studiare la difficoltà di $\text{PATH}^{\text{É}}(G, v, u, k)$ ci dà informazioni anche sulla difficoltà di $\text{MIN-PATH}(G, v, u)$

In questo caso vale anche il contrario. Se sappiamo risolvere $\text{PATH}^{\text{É}}(G, v, u, k)$ per ogni k, allora possiamo definire il seguente algoritmo per $\text{MIN-PATH}(G, v, u)$:

> Risolvere $\text{PATH}^{\text{É}}(G, v, u, k)$ per valori di k crescenti da 0 a $|V|-1$. Il risultato desiderato è il più piccolo k per cui otteniamo risposta SI per il problema di decisione. (sono sufficienti $\log(|V|)$ domande a $\text{PATH}^{\text{É}}(G, v, u, k)$)

Se consideriamo, invece, il problema di ottimizzazione:

$\text{MAX-PATH}(G, v, u)$: Dato un grafo $G = \langle V, E \rangle$ e due vertici v e u calcolare la lunghezza del massimo percorso semplice da v a u in G.

Il corrispondente problema di decisione è:

$\text{PATH}^{\text{É}}(G, v, u, k)$: Dato un grafo $G = \langle V, E \rangle$, due vertici v e u e un intero k, esiste un percorso da v a u in G con almeno k archi?

MISURA DELLA DIMENSIONE DI ISTANZE DI PROBLEMI

La complessità di un algoritmo in tempo o in spazio è misurata come funzione della dimensione dell'input, ovvero l'input è una qualche codifica dell'istanza corrente del problema che si vuole risolvere.
La codifica per un problema X è una funzione c che associa ad ogni istanza $x \in X$ una rappresentazione concreta $c(x)$ di x che può essere elaborata da un calcolatore.

ESEMPI:
Una istanza del problema del calcolo del K-esimo Numero di Fibonacci è costituita dal numero K la cui rappresentazione standard in informatica è una stringa binaria di lunghezza $\log_2 K$.
Una istanza del problema HAM-CYCLE è costituita da un grafo $G=(V,E)$, la cui rappresentazione può essere una matrice di adiacenza o un insieme di liste di adiacenza, entrambe di dimensione non superiore a $|V|^2$. In ultima analisi, però, entrambe queste 2 rappresentazioni vengono codificate come stringhe di bit di lunghezza $O(|V|^2)$.

La complessità di un algoritmo o di un problema può dipendere dal tipo di codifica scelta.

ESEMPIO: Sia dato un algoritmo per il calcolo dell'm-esimo Numero di Fibonacci impiega Tempo lineare sul valore m.
Consideriamo le seguenti codifiche:

a) m rappresentato in forma di stringa unaria
$m=100 \qquad c_1(m) = \overbrace{1111\cdots1}^{m}$

b) m rappresentato in forma di stringa binaria
$m=100 \qquad c_2(m) = 01100100$

c) m rappresentato in forma di stringa decimale
$m=100 \qquad c_3(m) = 100$

In base alla rappresentazione dell'input quanto tempo impiega l'algoritmo in funzione della dimensione della codifica dell'input?

$k=|c_1(m)| = m \implies T(k) = \Theta(k)$

$k=|c_2(m)| = \log_2 m \implies m = 2^k \in T(k) = \Theta(2^k)$

$k=|c_3(m)| = \log_{10} m \implies m = 10^k \in T(k) = \Theta(10^k)$

Con entrambe le codifiche c_2 e c_3, l'algoritmo impiega tempo esponenziale.

Assumeremo, d'ora in poi, sempre una codifica dell'input come stringa binaria.

ALGORITMI DI SOLUZIONE VERSUS ALGORITMI DI VERIFICA

Dato un problema di decisione X, un algoritmo di soluzione per X decide, per ogni istanza $x \in X$, se x è una istanza SI o NO per il problema.
Un algoritmo di verifica per X, invece, verifica se un qualche "certificato" c fornito in ingresso insieme all'istanza $x \in X$ dimostra che l'istanza $x \in X$ è un'istanza SI del problema X.

Per esempio, dato il problema

PATH^E(G, n, U, K): Dato un grafo $G=(V,E)$, due vertici n e u e un intero K, esiste un percorso da n a u in G con al più K archi?

Un algoritmo di verifica per PATH^E prenderebbe in ingresso $\langle G, n, U, K\rangle$ e una sequenza π di vertici e verificherebbe se le seguenti condizioni:

a) π è un percorso in G
b) π va da n a u
c) π contiene non più di K archi

Se a), b) e c) sono tutte e 3 vere, allora π è un certificato corretto per PATH^E(G, n, U, K) e quindi, l'algoritmo risponde che $\langle G, n, U, K\rangle$ è una istanza SI di PATH^E.
Altrimenti π non è un certificato corretto. In questo caso, non può dire se $\langle G, n, U, K\rangle$ è una istanza SI o una istanza NO di PATH^E.

In genere, decidere se un certificato è corretto o meno è più facile che decidere se $x \in X$ è una istanza SI o NO.

ESEMPIO
- Dato un grafo G, un ciclo Hamiltoniano è un ciclo semplice (nessuna ripetizione tranne primo vertice che è uguale all'ultimo) che visita tutti i vertici di G.
- Un grafo G si dice grafo Hamiltoniano se contiene un ciclo Hamiltoniano.

HAM-CYCLE(G): G è un grafo Hamiltoniano?

HAM-CYCLE ammette un algoritmo di verifica polinomiale.
Un certificato c per HAM-CYCLE(G) è una sequenza di vertici $c = \pi_0, \pi_1, \ldots, \pi_m$ (con $m = |V|$ e $G=(V,E)$). E' facile definire un algoritmo che in tempo polinomiale su $|G|$ e $|G|$ verifica se:

a) c è un percorso in G
b) $\pi_0 = \pi_m$
c) i primi m vertici in c sono tutti tra loro diversi

Se tutte e 3 le verifiche hanno successo, allora c è un ciclo Hamiltoniano in G e, quindi, G è una istanza SI del problema HAM-CYCLE.

LE CLASSI P E NP

La classe P è la classe dei problemi per cui esiste un algoritmo di soluzione, che impiega Tempo polinomiale $O(n^k)$, con $k \geq 0$ e n pari alla dimensione dell'istanza in esame.

La classe NP è la classe dei problemi per cui esiste un algoritmo di verifica, che impiega Tempo polinomiale $O(n^k)$, con $k \geq 0$ e n pari alla dimensione dell'istanza in esame.

Formalmente, $X \in NP$ se esiste un algoritmo $B_X(c,y)$ da esecito in ingresso 2 parametri, $x \in X$ e c, tale che:
- B_X impiega Tempo polinomiale
- per ogni $x \in X$, x è una istanza SI per X se e soltanto se esiste un certificato c, con $|c| \leq n^k$, $k \geq 0$ e $n = |x|$, tale che $B_X(x,c) = SI$.

P è la classe di problemi risolvibili in tempo polinomiale, NP è la classe dei problemi verificabili in tempo polinomiale.
L'esistenza del verificatore $B_X(\cdot,\cdot)$ permette di risolvere il problema X generando, dato $x \in X$, tutti i possibili certificati c, con $|c| \leq |x|^k$, ed eseguendo $B_X(x,c)$ per ciascun certificato. Ne risulta un algoritmo di "forza bruta" che impiega tempo esponenziale.

TEOREMA: $P \subseteq NP$

Dim
Sia $X \in P$. Allora esiste un algoritmo A_X che risolve qualsiasi istanza $x \in X$ in Tempo polinomiale rispetto a $|x|$.
Possiamo definire un verificatore $B(\cdot,\cdot)$ che, dati in ingresso x e c, ignora c ed esegue A_X su input x.
Se $A_X(x)$ risponde SI, $B(x,c)$ risponde SI, altrimenti $B(x,c)$ risponde NO. $B(x,c)$ risponde SI se e soltanto se x è un'istanza SI per X e impiega Tempo polinomiale, poiché A_X impiega Tempo polinomiale.

DIFFICOLTÀ RELATIVA E RIDUZIONI

Quando possiamo dire che un problema X è più difficile di un problema Y?

Dati 2 problemi X e Y scriviamo

$$Y \leq_p X$$

per indicare che X è difficile almeno quanto Y.

$Y \leq_p X$ si legge

"Y è riducibile a X in tempo polinomiale"

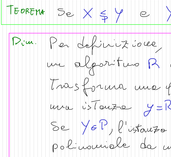
Dove:

Y è riducibile a X se esiste un algoritmo R, che trasforma una qualsiasi istanza y del problema Y in una istanza $R(y)$ del problema X, tale che le soluzioni di $R(y)$ e di y coincidono.

Se R impiega tempo polinomiale sulla dimensione di y per trasformare y in $R(y)$ allora R è una **riduzione polinomiale**.

TEOREMA Se $X \leq_p Y$ e $Y \in P$, allora $X \in P$.

Dim. Per definizione, se $X \leq_p Y$, allora esiste un algoritmo R di tempo polinomiale trasforma una qualsiasi istanza $x \in X$ in una istanza $y = R(x) \in Y$, tale che $Sol(x) = Sol(y)$. Se $Y \in P$, l'istanza $R(x)$ è risolvibile in tempo polinomiale da un algoritmo A_Y. Quindi:



$O(n^k)$ $\quad [n = |x| \text{ e } u = \max(|R(x)|, |x|)]$

A_X è un algoritmo che risolve in tempo polinomiale ogni istanza $x \in X$ eseguendo prima R e poi l'algoritmo A_Y che risolve Y.

Quindi $X \in P$ ■

Il contrapposto del teorema ci dice che:

Se $X \leq_p Y$ e $X \notin P$, allora $Y \notin P$.

Se possiamo ridurre in tempo polinomiale un problema X non risolvibile in tempo polinomiale a un dato problema Y, allora anche Y non è risolvibile in tempo polinomiale.

ESEMPIO:

Dato un grafo $G=(V,E)$, un **vertex cover** di G è un insieme di vertici $V' \subseteq V$ t.c. per ogni $(v,u) \in E$, $v \in V'$ oppure $u \in V'$. (V' copre ogni arco di G).

VERTEX-COVER (G,k): G contiene un vertex cover di dimensione al più k $(\leq k)$

Dato un grafo $G=(V,E)$, un **independent set** di G è un insieme di $V' \subseteq V$ t.c. per ogni $u, v \in V'$, $(u,v) \notin E$. (V' in contiene vertici connessi da archi di G)

INDEPENDENT-SET (G,k): G contiene un **independent set** di dimensione al più k $(\leq k)$

TEOREMA. Valgono entrambe le seguenti affermazioni:

VERTEX-COVER $\leq_p$ INDEPENDENT-SET

e

INDEPENDENT-SET $\leq_p$ VERTEX-COVER

Dim.

Consideriamo un grafo arbitrario $G=(V,E)$.

$V' \subseteq V$ è un **independent set** se e solo se $V \setminus V'$ è un **vertex cover**.

Se V' è un **independent set** e $(u,v) \in E$, allora almeno uno tra u e v non appartiene a V'. Quindi almeno uno tra u e v appartiene a $V \setminus V'$. Siccome ogni arco in E ha almeno uno dei suoi estremi in $V \setminus V'$, E è quindi un **vertex cover** di G.
Viceversa, se $V \setminus V'$ è un **vertex cover**, allora ogni arco $(u,v) \in E$ ha almeno uno dei 2 estremi in $V \setminus V'$. Tra quindi nessuna coppia di vertici in V' può essere connessa da un arco in E. Segue che V' è un **independent set** di G.

Quindi, se abbiamo un algoritmo A_{IS} per risolvere INDEPENDENT-SET possiamo risolvere VERTEX-COVER:
Data una istanza $\langle G,k \rangle$ per VERTEX-COVER è sufficiente costruire l'istanza $\langle G, |V|-k \rangle$ per INDEPENDENT-SET e risolverla tramite A_{IS}. Se A_{IS} restituisce SI allora $\langle G,k \rangle \in$ VERTEX-COVER. Poiché $\langle G, |V|-k \rangle$ può essere banalmente costruita in tempo polinomiale a partire da $\langle G,k \rangle$, otteniamo

VERTEX-COVER $\leq_p$ INDEPENDENT-SET

In maniera analoga (duale) si dimostra che

INDEPENDENT-SET $\leq_p$ VERTEX-COVER

■

TEOREMA Se $X \leq_p Y$ e $Y \leq_p Z$, allora $X \leq_p Z$

Dim. Siano dati un algoritmo R_{XY}, che trasforma una qualsiasi istanza $x \in X$ in una istanza $R_{XY}(x) \in Y$, e un algoritmo R_{YZ} che trasforma un y qualsiasi istanza $y \in Y$ in una istanza $R_{YZ}(y) \in Z$.
Allora possiamo definire un algoritmo R_{XZ} che, dato in ingresso una qualsiasi istanza $x \in X$, prima esegue l'algoritmo R_{XY} per ottenere $\bar{y} = R_{XY}(x)$ in tempo polinomiale su $|x|$ e poi esegue l'algoritmo R_{YZ} su $\bar{y}$ ottenendo $\bar{z} = R_{YZ}(\bar{y})$ in tempo polinomiale su $|\bar{y}|$. Poiché $|\bar{y}|$ è polinomiale su $|x|$ (perché?), $\bar{z}$ è ottenuto in tempo polinomiale su $|x|$. ■

PROBLEMI NP-COMPLETI

Un problema X si dice **NP-COMPLETO** se:

1) $X \in NP$
2) $\forall\, Y \in NP,\ Y \leq_p X$ (NP-DIFFICILE)

La classe dei problemi NP-COMPLETI è indicata con NPC.

Intuitivamente, i problemi NP-COMPLETI rappresentano i problemi più difficili della classe NP.

Se esistesse un algoritmo capace di risolvere un qualsiasi problema in NPC, allora tutti i problemi in NP sarebbero risolvibili in tempo polinomiale (per riduzione), e quindi varrebbe $P = NP$.

A tutt'oggi non è noto alcun algoritmo in grado di risolvere un qualche problema in NPC in tempo polinomiale.

LE CLASSI P E NP

La classe **P** è la classe dei problemi per cui esiste un **algoritmo di soluzione**, che impiega tempo polinomiale $O(n^k)$, con $k \geq 0$ e n pari alla dimensione dell'istanza in esame.

La classe **NP** è la classe dei problemi per cui esiste un **algoritmo di verifica**, che impiega tempo polinomiale $O(n^k)$, con $k \geq 0$ e n pari alla dimensione dell'istanza in esame.

Formalmente, $X \in NP$ se esiste un algoritmo $B_X(\cdot,\cdot)$ che accetta in ingresso 2 parametri, $x \in X$ e c, tale che:

- B_X impiega tempo polinomiale
- per ogni $x \in X$, x è una istanza SI per X se e soltanto se esiste un certificato c, con $|c| \leq n^k$, $k \geq 0$ e $n = |x|$, tale che $B_X(x,c) = SI$.

P è la classe di problemi risolvibili in tempo polinomiale, NP è la classe dei problemi verificabili in tempo polinomiale.

L'esistenza del verificatore $B_X(\cdot,\cdot)$ permette di risolvere il problema X generando, dato $x \in X$, tutti i possibili certificati c, con $|c| \leq |x|^k$, ed eseguendo $B_X(x,c)$ per ciascun certificato. Ne risulta un algoritmo di "**forza bruta**" che impiega **tempo esponenziale**.

TEOREMA. $P \subseteq NP$

Dim. Se $X \in P$. Allora esiste un algoritmo A_X che risolve ogni istanza $x \in X$ in tempo polinomiale rispetto a $|x|$. Possiamo definire un verificatore $B_X(\cdot,\cdot)$ che, dati in ingresso $x \in X$, qualsiasi c ed eseguito A_X con input x:

- $A_X(x)$ risponde SI, $B(x,c)$ risponde SI, altrimenti
- $B(x,c)$ risponde NO
- $B(x,c)$ risponde SI se e soltanto se x è un'istanza SI per X e impiega tempo polinomiale, poiché A_X impiega tempo polinomiale.

■

DIFFICOLTÀ RELATIVA E RIDUZIONI

Quando possiamo dire che un problema X è più difficile di un problema Y?

Dati 2 problemi X e Y scriviamo

$$Y \leq_P X$$

per indicare che X è difficile almeno quanto Y.

$$Y \leq_P X \quad \text{si legge}$$

"Y è riducibile a X in tempo polinomiale"

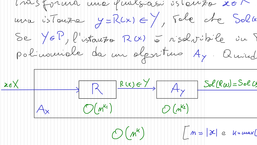
Dove:

Y è riducibile a X se esiste un algoritmo R, che trasforma una qualsiasi istanza y del problema Y in una istanza $R(y)$ del problema X, tale che la soluzione di $R(y)$ e di y coincidano.

Se R impiega tempo polinomiale sulla dimensione di y per trasformare y in $R(y)$ allora R è una **riduzione polinomiale**.

TEOREMA. Se $X \leq_P Y$ e $Y \in P$, allora $X \in P$.

Dim. Per definizione, se $X \leq_P Y$, allora esiste un algoritmo R che in tempo polinomiale trasforma una qualsiasi istanza $x \in X$ in una istanza $y = R(x) \in Y$, tale che $Sol(x) = Sol(y)$. Se $Y \in P$, l'istanza $R(x)$ è risolvibile in tempo polinomiale da un algoritmo A_Y. Quindi:



$O(n^k)$ $\quad [m = |x| \text{ e } u = \max(n,m)]$

A_X è un algoritmo che risolve in tempo polinomiale ogni istanza $x \in X$ eseguendo prima R e poi l'algoritmo A_Y che risolve Y.

Quindi $X \in P$

■

Il contrapposto del Teorema ci dice che:

Se $X \leq_P Y$ e $X \notin P$, allora $Y \notin P$.

Se possiamo ridurre in tempo polinomiale un problema X **non risolvibile** in tempo polinomiale a un dato problema Y, allora anche Y **non è risolvibile in tempo polinomiale**.

Esempio:

Dato un grafo $G=(V,E)$, un **vertex cover** di G è un insieme di vertici $V' \subseteq V$ t.c. per ogni $(v,w) \in E$, $v \in V'$ oppure $w \in V'$. (V' copre tutti gli archi di G).

VERTEX-COVER (G,k): G contiene un vertex cover di dimensione al più k ($\leq k$)

Dato un grafo $G=(V,E)$, un **independent set** di G è un insieme di vertici $V' \subseteq V$ t.c. per ogni $v,w \in V'$, $(v,w) \notin E$. (V' non contiene vertici connessi da archi di G)

INDEPENDENT-SET (G,k): G contiene un independent set di dimensione almeno k ($\geq k$)

TEOREMA. Valgono entrambe le seguenti affermazioni:

VERTEX-COVER $\leq_P$ INDEPENDENT-SET

e

INDEPENDENT-SET $\leq_P$ VERTEX-COVER

Dim. Consideriamo un grafo arbitrario $G=(V,E)$.

È facile osservare che un sottoinsieme di vertici $V \subseteq V$ è un **independent set** se e solo se $V \setminus V'$ è un **vertex cover**.

Se V' è un **independent set** e $(v,w) \in E$, allora almeno uno tra v e w **non appartiene a** V'. Quindi almeno uno tra v e w **appartiene a** $V \setminus V'$. Segue che ogni arco in E ha almeno uno dei suoi due estremi in $V \setminus V'$, che è quindi un **vertex cover** di G.

Viceversa, se $V \setminus V'$ è un **vertex cover**, allora ogni arco $(v,w) \in E$ ha almeno uno dei 2 estremi in $V \setminus V'$. Ma quindi nessuna coppia di vertici in V' può essere connessa da un arco in E. Segue che V' è un **independent set** di G.

Quindi, se abbiamo un algoritmo A_{IS} per risolvere INDEPENDENT-SET possiamo risolvere VERTEX-COVER.

Data una istanza $<G,k>$ per VERTEX-COVER è sufficiente costruire l'istanza $<G,|V|-k>$ per INDEPENDENT-SET e risolverla tramite A_{IS}. Se A_{IS} restituisce SI allora $<G,k>$ è una istanza SI di VERTEX-COVER. Infatti $<G,|V|-k>$ può essere banalmente costruita in tempo polinomiale a partire da $<G,k>$, ottenendo

VERTEX-COVER $\leq_P$ INDEPENDENT-SET

In maniera analoga (duale) si dimostra che

INDEPENDENT-SET $\leq_P$ VERTEX-COVER

■

TEOREMA Se $X \leq_P Y$ e $Y \leq_P Z$, allora $X \leq_P Z$

Dim. Siano dati un algoritmo R_{XY}, che trasforma una qualsiasi istanza $x \in X$ in una istanza $R_{XY}(x) \in Y$, e un algoritmo R_{YZ} che trasforma un a qualsiasi istanza $y \in Y$ in una istanza $R_{YZ}(y) \in Z$. Allora possiamo definire un algoritmo R_{XZ} che, dato in ingresso una qualsiasi istanza $x \in X$, prima esegue l'algoritmo R_{XY} per ottenere $\bar{y} = R_{XY}(x)$ in tempo polinomiale su $|x|$ e poi esegue l'algoritmo R_{YZ} su $\bar{y}$, ottenendo $\bar{z} = R_{YZ}(\bar{y})$ in tempo polinomiale su $|\bar{y}|$. Poiché $|\bar{y}|$ è polinomiale su $|x|$ (perché?), $\bar{z}$ è ottenuto in tempo polinomiale su $|x|$.

■

PROBLEMI NP-COMPLETI

Un problema X si dice **NP-COMPLETO** se:

1) $X \in NP$
2) $\forall\, Y \in NP,\ Y \leq_p X$ (NP-DIFFICILE)

La classe dei problemi NP-COMPLETI è indicata con NPC.

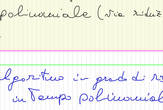
Intuitivamente, i problemi NP-COMPLETI rappresentano i problemi più difficili della classe NP.

Se esistesse un algoritmo in grado di risolvere un qualsiasi problema in NPC, allora tutti i problemi in NP sarebbero risolvibili in tempo polinomiale (via riduzione), e quindi sarebbe $P = NP$.

A tutt'oggi non è noto alcun algoritmo in grado di risolvere un qualche problema in NPC in tempo polinomiale.

SODDISFACIBILITÀ DEI CIRCUITI

PORTE LOGICHE



Ogni filo trasporta un valore binario (0/1).

x	NOT
0	1
1	0

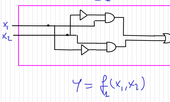
x_1	x_2	AND
0	0	0
0	1	0
1	0	0
1	1	1

x_1	x_2	OR
0	0	0
0	1	1
1	0	1
1	1	1

Ogni porta logica calcola una funzione Booleana specifica

Un CIRCUITO COMBINATORIO è un insieme di porte logiche tra loro connesse da fili.

La dimensione di un circuito C, indicata con $|C|$, è definita come la somma del n° di fili e del n° di porte logiche che C contiene.



C_2 calcola la funzione Booleana $f_2(x_1, x_2) = (x_1 \wedge \neg x_2) \vee (\neg x_1 \wedge x_2)$

$y = f_2(x_1, x_2)$

Un circuito C si dice **SODDISFACIBILE** se esistono valori Booleani per i fili di input tali che il filo di output produce valore 1.

Ad esempio il circuito C_2 nella figura sopra è soddisfacibile: dati in ingresso $x_1 = 1$ e $x_2 = 0$ (o $x_1 = 0$ e $x_2 = 1$) restituisce $y = 1$.

C_2 calcola la funzione Booleana $f_2(x_1, x_2)$ $\Rightarrow$

x_1	x_2	f_2
0	0	0
0	1	1
1	0	1
1	1	0

Un circuito C è soddisfacibile se la funzione Booleana che calcola è diversa dalla funzione costante 0.

CIRCUIT-SAT: Decidere se il circuito C in ingresso è soddisfacibile

TEOREMA. CIRCUIT-SAT $\in$ NP

Dim. Dato un circuito C, un certificato è un assegnamento x di valori 0/1 a ogni filo del circuito.
Chiaramente $|x| \leq |C|$, quindi la dimensione del certificato è polinomiale su $|C|$. Verificare che x è un certificato corretto può essere facilmente effettuato in tempo polinomiale su $|C|$, valutando se il valore del filo di output di ogni porta logica di C indicato nel certificato è identico al valore che quella porta logica produce quando riceve in ingresso i valori dei fili di input nel certificato. Ognuna di tali verifiche impiega tempo lineare sul n° di fili in ingresso alla porta, tale n° è $\leq |C|$.
Il n° di porte di C è chiaramente $\leq |C|$.
Quindi, il tempo richiesto è polinomiale su $|C|$. ∎

TEOREMA. CIRCUIT-SAT $\in$ NPC

Dim. 1) CIRCUIT-SAT $\in$ NP

Il Teorema precedente ci dà l'appartenenza a NP.

Per ottenere il risultato serve far vedere che ogni problema $X \in NP$ può essere ridotto a CIRCUIT-SAT.

2) CIRCUIT-SAT è NP-DIFFICILE

Si deve dimostrare che dato $x \in X$ esiste un circuito C_x, costruibile in tempo polinomiale su $|x|$, tale che:

x è un'istanza SI SSE C_x è soddisfacibile (1)

Dato $X \in NP$, esiste un algoritmo polinomiale A_X t.c. per ogni istanza $x \in X$, vale che

x è istanza SI SSE $\exists c$ $A_X(x, c) = 1$ (2)

Chiaramente $|c|$ è polinomiale su $|x| = n$ poiché in tempo polinomiale $t(n)$ si possono leggere (o scrivere o stampare) al massimo un numero di simboli polinomiali. Quindi $|c| \leq t(n)$.
Assumiamo una codifica binaria degli input e, in particolare, di c.
La dimostrazione costruisce un circuito C_x con input stringhe di bit di lunghezza $|c|$, tale che

$C_x(c) = 1$ SSE $A_X(x, c) = 1$ (3)

È facile osservare che (1) + (2) + (3) implicano la tesi.
Non è difficile osservare che, una volta fissati X e x, $A_X(x, c)$ calcola una funzione Booleana di $k = |c|$ variabili Booleane (una per ogni bit di c).
Quindi, una volta fissata l'istanza x, il comportamento di $A_X(x, c)$ può essere simulato da un circuito combinatorio con $|c|$ fili di input.

[ogni funzione Booleana su k variabili è calcolabile da un circuito con k fili di input]

Si può dimostrare che la dimensione di C_x può essere mantenuta polinomiale se $|c| \leq t(n)$, quindi, complessivamente polinomiale su $|x| = n$.

Assumo allora ottenuto un circuito C_x che produce 1 (cioè è soddisfacibile) se e soltanto se esiste una stringa di bit c t.c. $A_X(x, c) = 1$, il che dimostra (3). ∎

SODDISFACIBILITÀ BOOLEANA

Siano date n **variabili booleane** $X=\{x_1, x_2, \dots x_n\}$.

Un **Termine** booleano su X è una delle variabili x_i o la sua negazione $\bar{x}_i$ $(i \geq 1 \wedge i \leq n)$. I termini sono anche detti **letterali**.

Una **clausola** è semplicemente una **disgiunzione** di Termini su X

$$t_1 \vee t_2 \vee \dots \vee t_\ell$$

con $t_i \in \{x_1, \dots x_n, \bar{x}_1, \dots \bar{x}_n\}$

Una **formula** booleana in formato **CNF** (CONJUNCTIVE NORMAL FORM) è una congiunzione di clausole

$$C_1 \wedge C_2 \wedge \dots \wedge C_K$$

Una **formula CNF** è **soddisfacibile** se esiste un **assegnamento** di valori di verità alle variabili in X tale che la **formula** valuta a 1.

SAT: Data una formula CNF ϕ in ingresso, decidere se ϕ è soddisfacibile.

Una **formula** booleana in formato **3-CNF** è una congiunzione di clausole

$$C_1 \wedge C_2 \wedge \dots \wedge C_K$$

Tali che ogni clausola C_i contiene esattamente 3 Termini

$$C_i = t_i^1 \vee t_i^2 \vee t_i^3$$

3-SAT: Data una formula 3-CNF ϕ in ingresso, decidere se ϕ è soddisfacibile.

TEOREMA. 3-SAT $\in$ NPC

Dim. 1) 3-SAT $\in$ NP

Data una formula 3-CNF ϕ, un certificato è un assegnamento $\sigma: X \to \{0,1\}$, che associa un valore booleano a ogni variabile di ϕ.
Un algoritmo $A(\phi, \sigma)$ semplicemente sostituisce in ϕ ad ogni variabile x_i il valore $\sigma(x_i)$

ESEMPIO:

$\phi = (\bar{x}_1 \vee x_2 \vee \bar{x}_3) \wedge (x_1 \vee \bar{x}_4 \vee \bar{x}_5) \wedge (x_3 \vee \bar{x}_5 \vee x_6) \wedge (x_2 \vee x_5 \vee x_6)$

$\sigma = (x_1 \leftarrow 0, x_2 \leftarrow 0, x_3 \leftarrow 1, x_4 \leftarrow 1, x_5 \leftarrow 0, x_6 \leftarrow 0)$

$\Downarrow$

$\phi_\sigma = (1 \vee 0 \vee 0) \wedge (0 \vee 1 \vee 0) \wedge (0 \vee 1 \vee 1) \wedge (0 \vee 0 \vee 1)$

Dopodiché, l'algoritmo $A(\phi, \sigma)$ verifica che in **ciascuna clausola** di ϕ_σ sia presente almeno un 1.

2) CIRCUIT-SAT $\leq_p$ 3-SAT

Definiamo una procedura che, dato in ingresso un circuito combinatorio C, costruisce una formula 3-CNF ϕ_C.
L'idea è di introdurre **una variabile booleana** per ogni **filo di input** e per ogni **filo di output** delle porte del circuito.
Costruiamo poi, per ogni **porta logica** di C, una formula che descrive il **comportamento** (semantica) di quella porta logica in termini delle relazioni tra i suoi **fili di input** e di **output**.

ESEMPIO:

RELAZIONE INPUT/OUTPUT DELLE PORTE		FORMULE CNF	
$y_3 = \bar{x}_2$	equivalente a	$(\bar{x}_2 \vee \bar{x}_2) \wedge (x_2 \vee x_2)$	ϕ_3
$y_1 = \bar{x}_1$	equivalente a	$(\bar{x}_1 \vee \bar{x}_1) \wedge (x_1 \vee x_1)$	ϕ_1
$x_5 = x_3 \wedge x_4$	equivalente a	$(x_3 \vee \bar{x}_5) \wedge (\bar{x}_3 \vee x_5) \wedge (\bar{x}_4 \vee \bar{x}_5)$	ϕ_5
$x_6 = x_1 \wedge x_4$	equivalente a	$(x_1 \vee \bar{x}_6) \wedge (\bar{x}_1 \vee x_6) \wedge (\bar{x}_4 \vee x_6)$	ϕ_6
$x_2 = x_5 \vee x_6$	equivalente a	$(\bar{x}_5 \vee x_2) \wedge (x_5 \vee \bar{x}_6) \wedge (x_6 \vee \bar{x}_2)$	ϕ_2

$\phi_C = \phi_3 \wedge \phi_1 \wedge \phi_5 \wedge \phi_6 \wedge \phi_2 \wedge x_2$ — FORMULA CNF CHE CODIFICA SODDISFACIBILITÀ DI C

In generale, le **regole di traduzione** sono le seguenti:

x_i —x_y	$\Longrightarrow$	$(\bar{x}_y \vee \bar{x}_i) \wedge (x_y \vee x_i)$
x_i, x_k —x_y	$\Longrightarrow$	$(x_y \vee \bar{x}_i \vee \bar{x}_k) \wedge (\bar{x}_y \vee x_i) \wedge (\bar{x}_y \vee x_k)$
x_i, x_k —x_y	$\Longrightarrow$	$(\bar{x}_y \vee x_i \vee x_k) \wedge (x_y \vee \bar{x}_i) \wedge (x_y \vee \bar{x}_k)$

ϕ_C è, dunque, la **congiunzione** delle formule ottenute. Poiché il **nº di variabili** aggiunte è pari al **nº di porte logiche** di C, che è $\leq |C|$, e ogni formula ϕ_y ha dimensione costante (indipendente da $|C|$), la dimensione della formula ϕ_C è $O(|C|)$. La formula ϕ_C può essere banalmente costruita in tempo polinomiale su $|C|$.
Dobbiamo dimostrare che:

ϕ_C è soddisfacibile $\iff$ C è soddisfacibile

$\Longrightarrow$ Se ϕ_C è **soddisfacibile**, esiste un **assegnamento** per le variabili booleane che rende **vera** ogni formula ϕ_y (perché ϕ_C è la congiunzione delle ϕ_y).
Ogni formula ϕ_y assicura che il valore della variabile aggiuntiva x_y è identico al valore di output della corrispondente **porta logica**. Quindi il circuito C produce in output lo stesso valore di verità della variabile booleana che rappresenta il **filo di output** di C. Tale variabile (x_2 nell'esempio sopra) deve essere impostata a 1 affinché ϕ_C sia soddisfatta, per cui è stata costruita ϕ_C. Quindi, se il circuito C riceve sui **fili di input** i valori indicati nell'assegnamento che soddisfa ϕ_C, produrrà il valore della variabile che corrisponde al **filo di output**, valore che è, appunto, 1.

$\Longleftarrow$ Se C è soddisfacibile, l'assegnamento ai **fili di input** può essere esteso alle **variabili booleane** di ϕ_C, valutando il valore prodotto da ciascuna **porta logica** di C e assegnando lo stesso valore alla corrispondente **variabile booleana** in ϕ_C.
L'assegnamento così ottenuto soddisfa ϕ_C.

ϕ_C non è in forma 3-CNF (ma è in forma CNF con al massimo 3 letterali per clausola).
Può facilmente essere messo in formato 3-CNF aggiungendo 2 variabili booleane, p e q, in più:

$$(x_i \vee x_j) \iff (x_i \vee x_j \vee p) \wedge (x_i \vee x_j \vee \bar{p})$$

$$(x_i) \iff (x_i \vee p \vee q) \wedge (x_i \vee p \vee \bar{q}) \wedge (x_i \vee \bar{p} \vee q) \wedge (x_i \vee \bar{p} \vee \bar{q})$$

Effettuando queste sostituzioni in ϕ_C, otteniamo una formula 3-CNF equivalente di **dimensione polinomiale** su $|C|$. Da ciò discende la tesi.

■

SODDISFACIBILITÀ BOOLEANA

Siano date n variabili booleane $X=\{x_1, x_2, \dots, x_n\}$.

Un **Termine** booleano su X è una delle variabili x_i o la sua negazione $\bar{x}_i$ (i.e., $\neg x_i$). [I Termini sono anche detti **Letterali**.]

Una **clausola** è semplicemente una **disgiunzione** di Termini su X:

$$(\ell_1 \vee \ell_2 \vee \dots \vee \ell_k)$$

con $\ell_i \in \{x_1, \dots, x_n, \bar{x}_1, \dots, \bar{x}_n\}$.

ESEMPIO: $(\bar{x}_1 \vee x_2 \vee \bar{x}_3)$

Una **formula** booleana in formato **CNF** (CONJUNCTIVE NORMAL FORM) è una congiunzione di clausole

$$C_1 \wedge C_2 \wedge \dots \wedge C_M$$

ESEMPIO: $(\bar{x}_1 \vee x_2 \vee \bar{x}_3) \wedge (x_1 \vee x_2 \vee \bar{x}_4) \wedge (\bar{x}_1 \vee x_2 \vee \bar{x}_3) \wedge (x_3 \vee x_4 \vee x_1)$

Una **formula CNF** è **soddisfacibile** se esiste un **assegnamento** di valori di verità alle variabili in X tale che la formula valuta a 1.

ESEMPIO:

$\phi = (\bar{x}_1 \vee x_2 \vee \bar{x}_3) \wedge (x_1 \vee x_2 \vee \bar{x}_4) \wedge (\bar{x}_1 \vee x_2 \vee \bar{x}_3) \wedge (x_3 \vee x_4 \vee x_1)$

ϕ è soddisfacibile, ad esempio dall'assegnamento

$\sigma = (x_1 \leftarrow 0, x_2 \leftarrow 0, x_3 \leftarrow 1, x_4 \leftarrow 1)$

SAT: Data una formula CNF ϕ in ingresso, decidere se ϕ è soddisfacibile.

Una **formula** booleana in formato **3-CNF** è una congiunzione di clausole

$$C_1 \wedge C_2 \wedge \dots \wedge C_M$$

Tali che ogni clausola C_i contiene esattamente 3 Termini

$$C_i = (\ell_i^1 \vee \ell_i^2 \vee \ell_i^3)$$

3-SAT: Data una formula 3-CNF ϕ in ingresso, decidere se ϕ è soddisfacibile.

TEOREMA. 3-SAT $\in$ NPC

Dim. 1) 3-SAT $\in$ NP

Data una formula 3-CNF ϕ, un certificato è un assegnamento $\sigma: X \to \{0,1\}$, che associa un valore booleano a ogni variabile di ϕ.
Un algoritmo di verifica $A(\phi, \sigma)$ semplicemente sostituisce in ϕ ad ogni variabile x_i il valore $\sigma(x_i)$

ESEMPIO:

$\phi = (\bar{x}_1 \vee x_2 \vee \bar{x}_3) \wedge (x_1 \vee x_2 \vee \bar{x}_4) \wedge (\bar{x}_1 \vee x_2 \vee \bar{x}_3) \wedge (x_3 \vee x_4 \vee x_1)$

$\sigma = (x_1 \leftarrow 0, x_2 \leftarrow 0, x_3 \leftarrow 1, x_4 \leftarrow 0)$

$\Downarrow$

$\phi_\sigma = (1 \vee 0 \vee 0) \wedge (0 \vee 1 \vee 0) \wedge (0 \vee 1 \vee 1) \wedge (0 \vee 0 \vee 1)$

Dopodiché, l'algoritmo $A(\phi, \sigma)$ verifica che in ciascuna clausola di ϕ_σ sia presente almeno un 1.

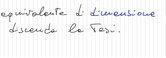
2) CIRCUIT-SAT $\leq_p$ 3-SAT

Definiamo una procedura che, dato in ingresso un circuito combinatorio C, costruisce una formula 3-CNF ϕ_C tale che C è soddisfacibile se e solo se lo è anche ϕ_C.

L'idea è di introdurre una variabile booleana per ogni filo di input e per ogni filo di output delle porte del circuito.

Costruiamo poi, per ogni porta logica di C, una formula che descrive il comportamento (semantica) di quella porta logica in termini delle relazioni tra i suoi fili di input e di output.

ESEMPIO:



RELAZIONE INPUT/OUTPUT DELLA PORTA

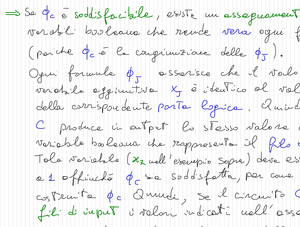
FORMULE CNF

$x_3 = \bar{x}_2$	equivalente a	$(\bar{x}_3 \vee \bar{x}_2) \wedge (x_3 \vee x_2)$	ϕ_3
$x_4 = \bar{x}_1$	equivalente a	$(\bar{x}_4 \vee \bar{x}_1) \wedge (x_4 \vee x_1)$	ϕ_4
$x_5 = x_3 \wedge x_4$	equivalente a	$(x_3 \vee \bar{x}_4 \vee \bar{x}_5) \wedge (\bar{x}_3 \vee x_4) \wedge (\bar{x}_3 \vee x_5)$	ϕ_5
$x_6 = x_1 \wedge x_5$	equivalente a	$(x_1 \vee \bar{x}_5 \vee \bar{x}_6) \wedge (\bar{x}_1 \vee x_5) \wedge (\bar{x}_1 \vee x_6)$	ϕ_6
$x_7 = x_6 \vee x_2$	equivalente a	$(\bar{x}_6 \vee x_7 \vee x_2) \wedge (x_6 \vee \bar{x}_7) \wedge (x_2 \vee \bar{x}_7)$	ϕ_7

$\phi_C \equiv \phi_3 \wedge \phi_4 \wedge \phi_5 \wedge \phi_6 \wedge \phi_7 \wedge x_7$

FORMULA CNF CHE CODIFICA SODDISFACIBILITÀ DI C

In generale, le **regole di traduzione** sono le seguenti:



ϕ_C è, dunque, la congiunzione delle formule ottenute partendo dal m^o di variabili aggiuntive è pari al m^o di porte logiche di C, che è $\leq |C|$, e ogni formula ϕ_T ha dimensione costante (indipendente da $|C|$), la dimensione della formula ϕ_C è $O(|C|)$. La formula ϕ_C può essere banalmente costruita in tempo polinomiale su $|C|$.
Dobbiamo dimostrare che:

$$\phi_C \text{ è soddisfacibile} \iff C \text{ è soddisfacibile}$$

$\Rightarrow$ Se ϕ_C è soddisfacibile, esiste un assegnamento per le variabili booleane che rende vera ogni formula ϕ_T (perché ϕ_C è la congiunzione delle ϕ_T).
Ogni formula ϕ_T assicura che il valore della variabile aggiuntiva x_T è identico al valore di output della corrispondente porta logica. Quindi, il circuito C produce in output lo stesso valore di verità della variabile booleana che rappresenta il filo di output di C. Tale variabile (x_7 nell'esempio sopra) deve essere impostata a 1 affinché ϕ_C sia soddisfatta, poiché è stata costruita ϕ_C. Quindi, se il circuito C viene sui fili di input i valori relativi all'assegnamento che soddisfa ϕ_C, produrrà il valore della variabile che corrisponde al filo di output, valore che è, appunto, 1. Quindi, C è soddisfacibile.

$\Leftarrow$ Se C è soddisfacibile, l'assegnamento ai fili di input può essere esteso alle variabili booleane di ϕ_C, valutando il valore prodotto da ciascuna porta logica di C e assegnando lo stesso valore alla corrispondente variabile booleana in ϕ_C.
L'assegnamento così ottenuto soddisfa chiaramente ϕ_C.

ϕ_C non è in forma 3-CNF (non è in forma CNF con al massimo 3 letterali per clausola)
Può facilmente essere riscritta in formato 3-CNF aggiungendo 2 variabili booleane, p e q, in più:

$$(x_i \vee x_j) \iff (x_i \vee x_j \vee p) \wedge (x_i \vee x_j \vee \bar{p})$$

$$(x_i) \iff (x_i \vee p \vee q) \wedge (x_i \vee \bar{p} \vee q) \wedge (x_i \vee p \vee \bar{q}) \wedge (x_i \vee \bar{p} \vee \bar{q})$$

Effettuando queste sostituzioni in ϕ_C, otteniamo una formula 3-CNF equivalente il cui numero polinomiale su $|C|$. Da ciò discende la tesi.

■

INDEPENDENT SET e VERTEX COVER

Dato un grafo $G=\langle V,E\rangle$, un **independent set** di G è un insieme di $V' \subseteq V$ t.c., per ogni $v,u \in V'$, $(v,u) \notin E$.

INDEPENDENT-SET(G,k): G contiene un independent set di dimensione almeno k ($\geq k$)

TEOREMA. INDEPENDENT-SET è NP-COMPLETO

Dim.

1) INDEPENDENT-SET $\in$ NP. Un certificato è un sottoinsieme $c \subseteq V$ di vertici. Verificare che $|c| \leq k$ e che ogni arco $(v,u) \in E$ ha almeno un estremo fuori da c può essere fatto in tempo $O(|E| \cdot |c|) = O(|E| \cdot |V|)$, quindi in tempo polinomiale sulla dimensione del grafo G ($|E| \cdot |V| \leq |G|^2$).

2) 3-SAT $\leq_p$ INDEPENDENT-SET

OSSERVAZIONE: Per soddisfare una formula 3-CNF si deve poter
1 - selezionare un letterale di ogni clausola
2 - trovare un assegnamento che renda veri tutti i letterali selezionati al punto 1.

2 è possibile solo se non si selezionano letterali opposti (x_i e $\bar{x}_i$)

Data la formula 3-CNF ϕ, costruiamo un grafo G_ϕ in cui:

- un gruppo di 3 vertici per ogni clausola C_i in ϕ che contiene i vertici $V^i = \{v_1^i, v_2^i, v_3^i\}$

$$V = \bigcup_{i=1}^{k} V^i$$

- ciascuno dei 3 letterali della clausola C_i etichetta uno dei vertici di V^i

$$C_i = (l_1^i \vee l_2^i \vee l_3^i) \implies lab(v_1^i) = l_1^i,\ lab(v_2^i) = l_2^i,\ lab(v_3^i) = l_3^i$$

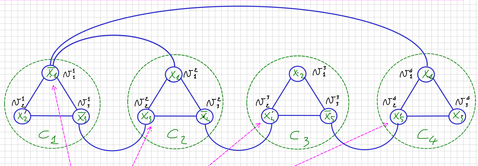
- un arco tra due vertici v e u di V se
  - v e u appartengono allo stesso gruppo V^i
  - v e u sono etichettati con letterali opposti (x_j e $\bar{x}_j$).

$$E = \{(v,u) \mid \{v,u\} \subseteq V^i \text{ per } i \in [k] \cup \{(v,u) \mid lab(v) = \overline{lab(u)}\}$$

ESEMPIO:

$$\phi = \underbrace{(\bar{x}_1 \vee x_2 \vee \bar{x}_3)}_{C_1} \wedge \underbrace{(x_1 \vee x_4 \vee \bar{x}_6)}_{C_2} \wedge \underbrace{(x_3 \vee x_4 \vee \bar{x}_5)}_{C_3} \wedge \underbrace{(x_2 \vee x_5 \vee x_3)}_{C_4}$$

G_ϕ



$I = \{v_1^1, v_2^2, v_2^3, v_3^4\}$ — I è independent set di dimensione 4

$\sigma_I = \langle x_2=0, x_4=1, x_3=1, x_5=1 \rangle$ — σ_I assegnamento che soddisfa ϕ

G_ϕ ha chiaramente dimensione polinomiale su $|\phi|$ e può essere costruito in tempo $\Theta(|G_\phi|)$.

G_ϕ ha un independent set di dimensione $\geq k$ se e solo se ϕ è soddisfacibile.

$\Rightarrow$ Se G_ϕ ha un independent set I di dimensione $\geq k$ definiamo l'assegnamento σ_I come segue:

$$\forall x \in X. \quad \sigma_I(x) = \begin{cases} 0 & \text{se } x \notin I \\ 1 & \text{se } x \in I \end{cases}$$

σ_I è un assegnamento che soddisfa ϕ. Infatti, I contiene un vertice per ogni clausola e non può contenere vertici etichettati con letterali opposti (x_i e $\bar{x}_i$). σ_I rende veri i letterali dei vertici in I, quindi soddisfa ciascuna clausola e, quindi, ϕ.

$\Leftarrow$ Se ϕ è soddisfacibile esiste un assegnamento σ che rende vero almeno un letterale per ogni clausola. Costruiamo I a partire da σ, selezionando un vertice in ciascun V^i etichettato con un letterale reso vero da σ. I ha dimensione k ed è un independent set di G_ϕ. Infatti, se così non fosse, ci dovrebbe essere un arco tra due vertici contenuti in I, siano essi v e u. Per come è stato costruito I, $v \in V^i$ e $u \in V^j$ con $i \neq j$ (v e u non possono appartenere allo stesso gruppo di vertici). Ma allora, se $(v,u) \in E$, v e u devono avere come etichetta due letterali opposti. Tali letterali non possono essere entrambi resi veri da σ. Quindi v e u non possono essere entrambi in I. Quindi, I è un independent set. ■

COROLLARIO. VERTEX-COVER è NP-COMPLETO

Dim. Poiché INDEPENDENT-SET $\leq_p$ VERTEX-COVER e poiché INDEPENDENT-SET $\in$ NPC.

PROBLEMA DELLO ZAINO 0/1

SUBSET-SUM (V, W): Dato un insieme di numeri naturali $V = \{n_1, \dots, n_n\}$ esiste un sottoinsieme di V la cui somma è pari a W?

TEOREMA. SUBSET-SUM è NP-COMPLETO

Dim.

1) SUBSET-SUM $\in NP$. Un certificato c è un sottoinsieme di numeri in V. Possiamo verificare che $c \in V$ e che la somma dei numeri in c è $= W$ in tempo polinomiale su $|V|$ e $|W|$.

2) 3-SAT $\leq_P$ SUBSET-SUM

Data una formula 3-CNF ϕ con variabili booleane $\{x_1, \dots x_n\}$ e k clausole, $\phi = C_1 \wedge C_2 \wedge \dots \wedge C_k$, costruiamo in tempo polinomiale un'istanza di SUBSET-SUM, seguendo la procedura delineata nel seguente esempio.

Consideriamo la formula 3-CNF

$$\phi = (x_1 \vee \bar{x}_2 \vee x_3) \wedge (x_2 \vee x_3 \vee x_1) \wedge (\bar{x}_3 \vee \bar{x}_1 \vee x_1) \wedge (x_1 \vee x_2 \vee \bar{x}_3)$$

Costruiamo una istanza P_ϕ del problema SUBSET-SUM come segue.

Introduciamo:

- 2 numeri per ogni **variabile**
  - v_{x_i}: per il letterale x_i
  - $v_{\bar{x}_i}$: per il letterale $\bar{x}_i$
- 2 numeri per ogni **clausola**
  - v_{c_j}: per la clausola C_j
  - v'_{c_j}: per la clausola C_j
- Ogni numero ha:
  - 1 cifra decimale per ogni variabile
  - 1 cifra decimale per ogni clausola

Le cifre dei vari numeri sono stabilite come segue:

- v_{x_i} e $v_{\bar{x}_i}$ hanno un 1 nella cifra x_i (colonna x_i) e in ogni cifra C_j t.c. x_i (rispettivamente $\bar{x}_i$) occorre nella clausola C_j. Le rimanenti cifre sono 0.
- v_{c_j} e v'_{c_j} hanno 0 ovunque, tranne nella cifra C_j, che contiene un 1.

Nota: La somma delle cifre in ogni colonna è al massimo 5. Nessuna somma di numeri richiede mai un riporto.

ϕ è soddisfacibile $\iff$ P_ϕ è un'istanza sì del problema SUBSET-SUM

$\Rightarrow$ Se ϕ è soddisfacibile, sia σ un assegnamento che rende ϕ vera. Costruiamo un certificato per P_ϕ come segue:

c contiene v_{x_i} se $\sigma(x_i) = 1$ e $v_{\bar{x}_i}$ se $\sigma(x_i) = 0$.
Quindi, per ogni variabile booleana x_i:

- Uno solo tra v_{x_i} e $v_{\bar{x}_i}$ viene selezionato in c

Poiché σ soddisfa ϕ, rende vero almeno un letterale per ogni clausola C_j.

Inoltre, per ciascuna clausola C_j, con $1 \leq j \leq k$:

- Se ϕ rende vero **solo un letterale** in C_j, vengono aggiunti a c sia v_{c_j} che v'_{c_j}.
- Se ϕ rende vero **2 letterali** in C_j, viene aggiunto a c solo v_{c_j}
- Se ϕ rende vero **3 letterali** di C_j, né v_{c_j} né v'_{c_j} fanno parte di c.

È facile osservare che la somma dei numeri inseriti in c risulta in un numero che ha tutte le cifre corrispondenti alle variabili uguali a 1 e tutte le cifre corrispondenti alle clausole uguali a 3. Quindi c è un certificato corretto per P_ϕ.

$\Leftarrow$ Dato un certificato corretto c per P_ϕ, allora uno solo tra v_{x_i} e $v_{\bar{x}_i}$ può essere presente in c. Quindi possiamo definire σ come segue:

$$\sigma(x_i) = \begin{cases} 1 & \text{se } v_{x_i} \in C \\ 0 & \text{se } v_{\bar{x}_i} \in C \end{cases}$$

Poiché la somma di valori in c deve essere uguale a W, ogni cifra corrispondente a una clausola in quella somma deve essere uguale a 3. Quindi, per ciascuna clausola C_j, almeno uno dei valori v_ℓ (con ℓ un letterale di ϕ) presente in c deve avere un 1 nella sua cifra C_j. Questo significa che quel letterale è preso vero da σ e rende soddisfatta la clausola C_j. Essendo ogni clausola soddisfatta da σ, ϕ è soddisfacibile. ∎

Il problema dello zaino 0/1 ha un analogo problema decisionale:

KNAPSACK: Dati in ingresso un insieme $I = \{1, 2, \dots, n\}$ di oggetti, ciascuno con peso w_i e valore v_i, un numero naturale W e un numero naturale K, esiste un sottoinsieme di oggetti di I con peso complessivo $\leq W$ e valore complessivo $\geq K$.

TEOREMA. KNAPSACK è NP-COMPLETO

Dim.

1) KNAPSACK $\in NP$ è lasciato come esercizio

2) SUBSET-SUM $\leq_P$ KNAPSACK

Date un'istanza di SUBSET-SUM (V, W) con $X = \{x_1, \dots, x_n\}$, definiamo una istanza di KNAPSACK come segue $(I, \{P_i\}, \{v_i\}, W, K)$ dove

$I = \{1, 2, \dots, n\}$	è l'insieme di oggetti
$P_i = x_i$	per ogni $1 \leq i \leq n$
$v_i = x_i$	per ogni $1 \leq i \leq n$
$K = W$	(valori e pesi coincidono)

È immediato verificare che:

esiste un sottoinsieme di X la cui somma è W.

se e soltanto se

esiste uno zaino di capacità massima W il cui valore è almeno $K = W$.

∎

Algoritmi di approssimazione

Per i problemi **non trattabili** è poco probabile (o impossibile) poter definire algoritmi efficienti in grado di risolvere in maniera esatta istanze di dimensione molto grande.

In questi casi ci si può spesso accontentare di ottenere **soluzioni sufficientemente vicine** a quella esatta. Se il problema è un problema di ottimizzazione si può provare a definire un algoritmo in grado di produrre una **soluzione subottima**, ma "abbastanza buona", **in tempo polinomiale**.

Come valutiamo se una soluzione è "abbastanza buona"?

Supponiamo che X sia un problema di ottimizzazione in cui vogliamo ottenere una soluzione che **minimizza** il costo e sia C^* il costo della soluzione ottima. Sia C il costo di una soluzione subottima. Chiaramente $C^* \leq C$

Il rapporto $\frac{C}{C^*}$, detto **rapporto di approssimazione**, misura quanto lontano è la soluzione subottima (approssimata) da quella ottima. Tale rapporto è sempre ≥ 1.

Se $\frac{C}{C^*}=1$ le due soluzioni hanno lo stesso costo e, quindi, sono entrambe ottime.

In generale, il rapporto di approssimazione può dipendere dalla dimensione $n=|x|$ dell'istanza $x \in X$ che vogliamo risolvere.

Data un'istanza $x \in X$ di un problema di **minimizzazione** X e una funzione $\rho(n)$, diciamo che un algoritmo è una **$\rho(n)$-approssimazione** se:

$$\frac{C}{C^*} \leq \rho(n)$$

Se X è un problema di **massimizzazione**, il rapporto di approssimazione è indicato con $\frac{C^*}{C}$, che è sempre ≥ 1.

In generale, dato un problema di ottimizzazione X e una funzione $\rho(n)$, un algoritmo per X si dice **$\rho(n)$-approssimato** se

$$\max\left(\frac{C}{C^*}, \frac{C^*}{C}\right) \leq \rho(n)$$

Ad esempio, se un algoritmo garantisce di restituire soluzioni di costo al più doppio (o almeno la metà) del costo della soluzione ottima, allora

$$\max\left(\frac{C}{C^*}, \frac{C^*}{C}\right) \leq 2$$

e l'algoritmo si dice **2-approssimato** ($\rho(n)=2$ in questo caso). La distanza dell'ottimo può dipendere dalla dimensione della istanza in input. Ad esempio, se

$$\max\left(\frac{C}{C^*}, \frac{C^*}{C}\right) \leq \log n$$

la soluzione approssimata si allontana sempre più dall'ottimo al crescere della dimensione del problema.
In questo caso, otteniamo una **$\log n$-approssimazione**.

Algoritmo 2-approssimato per Vertex Cover

Consideriamo il problema del **vertex cover** nella sua formulazione classica come problema di ottimizzazione.
Un **vertex cover** di un grafo non orientato $G=(V,E)$ è un sottoinsieme $V' \subseteq V$ tale che, per ogni arco $(u,v) \in E$, almeno uno tra u e v è incluso in V'.

MIN-VERTEX-COVER: dato un grafo non orientato $G=(V,E)$, restituire il **vertex cover** di **dimensione minima**.

Sappiamo che MIN-VERTEX-COVER è almeno tanto difficile quanto VERTEX-COVER che abbiamo dimostrato essere **NP-COMPLETO**

È, quindi, poco probabile che possa essere risolto in maniera esatta in tempo efficiente.

È, però, relativamente semplice calcolare un **vertex cover** di dimensione non troppo superiore all'ottimo.

L'idea è quella di calcolare un **vertex cover**, selezionando di volta in volta un arco $t \in E$ in modo casuale, aggiungere gli estremi ad un insieme inizialmente vuoto ed eliminare tutti gli archi del grafo incidenti nei due estremi appena aggiunti.

L'algoritmo risultante è:

```
APPROX_VERTEX_COVER(G)
  S = ∅ , A = ∅
  E' = E
  WHILE E' ≠ ∅ DO
      SCEGLIERE UN ARCO (u,v) ∈ E'
      A = A ∪ {(u,v)}
      S = S ∪ {u,v}
      E' = E' \ {(x,y) | x = u o y = v}
  RETURN S
```

Le istruzioni in rosso non sono necessarie ma ci aiutano nella dimostrazione.

APPROX_VERTEX_COVER(G) restituisce un **vertex cover** di G in **tempo polinomiale** su $|G|$.

Dim. Ad ogni iterazione, l'algoritmo seleziona un arco di E' ed elimina almeno un arco (quello selezionato) da E'. Quindi, il **n° di iterazioni** del WHILE è al più pari alla cardinalità iniziale di E', che è pari a $|E|$. Le operazioni di **rimozione** degli archi da E' possono essere eseguite in **tempo complessivo** per a $|E|$ (Perché?). Le operazioni di **aggiunta** di vertici a C possono essere eseguite complessivamente in tempo lineare su $|V|$.
Quindi, il tempo necessario è lineare su $|G|=|V|+|E|$.

Se S^* è il minimo vertex cover di G e S è il risultato restituito da APPROX_VERTEX_COVER(G), allora

$$|S| \leq 2 \cdot |S^*|$$

Dim. Consideriamo l'insieme A degli archi che vengono selezionati ad ogni ciclo WHILE. Sono gli archi inseriti nell'insieme A.
Osserviamo che gli archi che vengono inseriti in A sono archi che non condividono tra loro gli estremi. Cioè:

$$(u_1,v_1) \in A \text{ e } (u_2,v_2) \in A \implies \{u_1,v_1\} \cap \{u_2,v_2\} = \emptyset$$

Ciò è dovuto al fatto che ogni volta che un arco $e \in E$ viene inserito in A, tutti gli archi che hanno un estremo in comune con e vengono rimossi da E'.
Se C^* è un **vertex cover minimo** di G, allora C^* deve contenere almeno uno dei due estremi di ciascun arco di A, i quali non condividono estremi tra di loro.
Quindi vale necessariamente che

$$|C^*| \geq |A| \qquad (1)$$

L'insieme C restituito dall'algoritmo è l'unione degli estremi degli archi di A, che sono tutti distinti, quindi:

$$|C| = 2 \cdot |A| \qquad (2)$$

Da (1) e (2) otteniamo immediatamente che $|C| \leq 2 \cdot |C^*|$. ∎

APPROX_VERTEX_COVER è un algoritmo **2-approssimato** per MIN-VERTEX-COVER.

Algoritmo 2-approssimato per lo zaino 0/1

Sia dato un insieme di oggetti $I=\{1,2,\dots,n\}$ e un numero naturale W. L'oggetto i-esimo ha peso $w_i \le W$ e valore v_i. Assumiamo che I sia ordinato per valori decrescenti: $v_1 \ge v_2 \ge \dots \ge v_n$.
Sia W il peso massimo ammesso per un sottoinsieme di oggetti.

Sia I_J^{opt} il problema di trovare un zaino di capacità W ca oggetti dell'insieme $\{J, J+1, \dots, n\}$ in modo che

1) l'oggetto J-esimo sia contenuto nello zaino.
2) la somma dei valori degli oggetti contenuti sia massima.

Analogamente, II_J^{opt} il problema di trovare un zaino di capacità W ca porzioni di oggetti dell'insieme $\{J, J+1, \dots, n\}$ in modo che

1) l'oggetto J-esimo sia completamente contenuto nello zaino.
2) la somma dei valori delle porzioni degli oggetti contenuti sia massima.

---

a) La soluzione ottima S^* al problema (I, W) è la soluzione ottima di uno dei problemi I_J^{opt}, con $1 \le J \le n$. Cioè:

$$S^* \in \{P_1, P_2, \dots, P_n\}$$

dove P_J è soluzione ottima a I_J^{opt}, con $1 \le J \le n$.

Dim.

Sia $S^* = J_1 J_2 \dots J_k$ soluzione ottima per l'istanza (I, W) del problema dello zaino 0/1, ordinata in modo che

$$J_1 < J_2 < \dots < J_k$$

Chiaramente, S^* è anche una soluzione ammissibile per il problema $I_{J_1}^{opt}$, poiché contiene l'oggetto J_1 e un sottoinsieme di oggetti di $\{J_1, J_1+1, \dots, n\}$.
S^* è anche la soluzione ottima per $I_{J_1}^{opt}$, altrimenti, se ce ne fosse, esisterebbe una soluzione S' migliore per $I_{J_1}^{opt}$ che, ovviamente, sarebbe ammissibile per (I, W), contraddicendo l'ottimalità di S^* per (I, W).
Quindi, $S^* = P_{J_1}$ e la tesi è dimostrata.

---

b) La soluzione ottima Q_J al problema frazionario I_J^{fra} è ottenuta
- selezionando tutto l'oggetto J.
- applicando l'**algoritmo greedy** per lo zaino frazionario con input $\{J+1, \dots, n\}$ e $W - w_J$.

Dim.

La soluzione così ottenuta ha la forma $(J, w_J) (J_2, y_2) \dots (J_k, y_k)$ con

$$w_J + \sum_{\ell=1}^{k} y_\ell \le W$$

la sequenza $(J, 1) \dots (J_k, y_k)$ è soluzione ammissibile della istanza $(\{J+1, \dots, n\}, W - w_J)$ del problema dello zaino frazionario. Supponiamo che l'algoritmo greedy produca sempre una sol. ottima per l'istanza che riceve, e che la tesi.

---

c) Per ogni $1 \le J \le n$, Q_J contiene al più una sola porzione un completo di un qualche oggetto. Cioè ogni elemento in Q_J è del tipo (i, w_i), tranne al più un elemento (k, y_k) con $y_k < w_k$.

Dim.

Q_J ha la forma $(J, w_J) (J_2, y_2) \dots (J_k, y_k)$ per il fatto di ammissibilità del problema I_J^{fra}. Inoltre, supponi che $(J_2, y_2) \dots (J_k, y_k)$ è stata ottenuta utilizzando l'algoritmo greedy per lo zaino frazionario con input $(\{J+1, \dots, n\}, W - w_J)$.
La prima scelta greedy è quella di inserire la massima quantità possibile dell'oggetto con il massimo rapporto $\frac{v_i}{w_i}$ (valore su peso). Supponiamo che tale oggetto sia proprio J_2, allora abbiamo 2 casi:

1) $w_{J_2} > W - w_J$ e in tal caso $y_2 = W - w_J$ e $k = 1$ (non vi è null'altro zaino oltre a J e J_2). Poiché J è intero e J_2 è una porzione, la tesi è soddisfatta.
2) $w_{J_2} \le W - w_J$ e in tal caso $y_2 = w_{J_2}$. Quindi, sia J che J_2 sono presi per intero e $(J_3, y_3) \dots (J_k, y_k)$ è soluzione ottima di $(\{J_2+1, \dots, n\}, W - w_J - w_{J_2})$. Ripetendo lo stesso ragionamento per tutte le scelte greedy contenute in Q_J, arriviamo o **sempre** nel caso 2) o nel caso 2) fino alla $(k-1)$-esima scelta e nel caso 1) per la scelta k-esima. In conseguenza tutti gli elementi vengono presi per intero tranne al più l'ultimo.

---

d) Per ogni $1 \le J \le n$, sia $\hat{Q}_J$ la soluzione ottenuta **eliminando** da Q_J la **porzione non intera**. Allora, $\hat{Q}_J$ è soluzione ammissibile per il problema I_J^{opt} e

$$C(\hat{Q}_J) \ge \frac{C(P_J)}{2}$$

Dim. Ogni soluzione a un problema dello zaino 0/1 è ammissibile per il corrispondente problema frazionario (che ha meno vincoli).
Quindi vale certamente che:

$$C(Q_J) \ge C(P_J)$$

per ogni $1 \le J \le n$.
Preso un arbitrario $1 \le J \le n$, per la proprietà c) sappiamo che

$$Q_J = (J, w_J) (J_2, w_{J_2}) \dots (J_{k-1}, w_{J_{k-1}}) (J_k, y_k)$$

con $y_k < w_{J_k}$.
Quindi:

$$C(Q_J) = v_J + v_{J_2} + \dots + v_{J_{k-1}} + \frac{y_k}{w_{J_k}} v_{J_k} \qquad (1)$$

Poiché gli oggetti sono ordinati per valore decrescente,

$$v_J \ge v_{J_2} \quad \text{e, poiché } y_k < w_{J_k}, \quad v_{J_k} > \frac{y_k}{w_{J_k}} v_{J_k}$$

quindi

$$v_J \ge \frac{y_k}{w_{J_k}} v_{J_k}$$

Ma allora vale anche che

$$v_J + \sum_{\ell=1}^{k-1} v_{J_\ell} \ge \frac{y_k}{w_{J_k}} v_{J_k} \qquad (2)$$

Sommando $v_J + \sum_{\ell=1}^{k-1} v_{J_\ell}$ a entrambi i membri della disuguaglianza (2) otteniamo che

$$2\left(v_J + \sum_{\ell=1}^{k-1} v_{J_\ell}\right) \ge v_J + \sum_{\ell=1}^{k-1} v_{J_\ell} + \frac{y_k}{w_{J_k}} v_{J_k} \qquad (3)$$

Ma $\hat{Q}_J = (J, w_J) (J_2, w_{J_2}) \dots (J_{k-1}, w_{J_{k-1}})$ e quindi:

$$C(\hat{Q}_J) = v_J + \sum_{\ell=1}^{k-1} v_{J_\ell} \qquad (4)$$

Da (1), (3) e (4) otteniamo che

$$2\,C(\hat{Q}_J) \ge C(Q_J)$$

e, della (1) per transitività

$$2C(\hat{Q}_J) \ge C(P_J)$$

da cui discende immediatamente la tesi.

Algoritmo Approssimato

Consideriamo l'algoritmo $A(I, W)$ che per ogni $1 \le J \le n$:

- calcola la soluzione Q_J con l'**algoritmo greedy** per lo zaino frazionario,
- ottiene da Q_J la soluzione $\hat{Q}_J$ eliminando l'elemento frazionario,
- infine restituisce, tra $\hat{Q}_J$, quella di **valore massimo**.

Teorema. Se $S = A(I, W)$ e S^* è la soluzione ottima al problema dello zaino 0/1 con input (I, W), allora

$$C(S) \ge \frac{C(S^*)}{2}$$

Dim. Dalla proprietà d) sappiamo che $C(\hat{Q}_J) \ge \frac{C(P_J)}{2}$ per ogni $1 \le J \le n$. Dalla proprietà a) supponiamo che S^* è uno dei P_J. Infine S è il migliore tra i $\hat{Q}_J$. Mettendo insieme tutto ciò si ottiene proprio la tesi.

Quindi $A(I, W)$ è un algoritmo 2-approssimato per lo zaino 0/1.

È immediato osservare che il tempo richiesto da $A(I, W)$ è polinomiale su n, in particolare è $O(n^2)$, dove $n = |I|$ (quindi non dipende da W).